

封面

标题：《旅游模拟器 / Travel Simulator》V1 最终技术落地方案报告

副标题：Backend Architecture, Data System, Trip Simulation Engine & Economy Design

版本：Clean Final V1 Technical Design Report

日期：2026 年 5 月 15 日

受众：开发团队、后端工程师、客户端工程师、游戏策划、数据分析师、UI/UX 设计师、项目经理、投资/团队成员

文档项	说明
交付属性	可直接交付开发团队执行的 V1 技术落地方案，不是概念稿或聊天记录。
核心地基	Trip Simulation Engine / 旅程模拟引擎、Economy Ledger / 经济流水系统、Route Config System / 路线配置系统。
设计边界	V1 不做真实全球地图，不接 Google Maps，不做真实导航，不做复杂微服务，不把 Kubernetes 作为必需项。
后端原则	后端权威计算；客户端只负责表现、输入和提交玩家意图。
经济原则	所有 Road Coins、Travel Tokens、Souvenir Stamps、Stamp Fragments、Blueprints 的变化必须进入 wallet_transactions。
体验原则	核心循环是长途旅行、离线回归、照片收集、车辆维护和路线解锁；Gacha 只服务收藏与外观。

版本信息

字段	内容
文档名称	《旅游模拟器 / Travel Simulator》V1 最终技术落地方案报告
版本	Clean Final V1 Technical Design Report
日期	2026 年 5 月 15 日
文档目标	将原始 V1 最终方案清理为可落地、可开发、可测试、可运营、可投资阅读的正式技术文档。
覆盖范围	产品定位、玩家循环、客户端表现、后端架构、路线系统、经济系统、车辆维护、在线/离线模拟、Travel Report、数据库、API、反作弊、Admin Config、Analytics、测试、风险和路线图。
关键设计原则	V1 可落地；后端权威；数值可配置；经济可审计；离线收益克制；Gacha 不影响核心进度；路线解锁不变成 paywall；燃油不变成体力。
本次 Clean Version 优化说明	修复中文排版和断句；统一术语；统一默认经济数值；补充客户端主界面体验层；补充 V1 Scope Control；补充 offline_report 防重复生成规则；调整 Daily Login 的 Souvenir Stamp 风险；强化 P0/P1/P2、测试、风险和 checklist。

版本	日期	说明
Clean Final V1	2026 年 5 月 15 日	基于原始 PDF / DOCX 方案进行最终清理、统一、补强和正式排版。

目录

封面.....1

版本信息.....2

0. Executive Summary / 执行摘要.....9

1. V1 产品与技术定位.....9

 1.1 游戏定位.....9

 1.2 V1 核心目标.....10

 1.3 V1 非目标.....10

 1.4 V1 成功标准.....10

 1.5 V1 不做什么.....10

 1.6 技术设计原则.....10

2. 核心玩家循环.....11

 2.1 主循环.....11

 2.2 每日循环.....11

 2.3 离线回归循环.....11

3. V1 Client Gameplay Presentation / 主界面表现与玩家体验层.....11

 3.1 主屏幕驾驶表现.....11

 3.2 像素风 route segment 背景切换.....12

 3.3 天气/昼夜表现.....12

 3.4 Route Board.....12

 3.5 Travel Report UI.....12

 3.6 景点停靠与照片卡.....12

 3.7 客户端预测动画与后端权威状态.....12

4. 第一趟旅程教程系统.....12

 4.1 教程目标.....12

 4.2 教程状态机.....13

 4.3 教程数据字段.....13

 4.4 教程 API.....13

 4.5 教程测试重点.....14

 4.6 教程风险.....14

5. 总体技术架构.....14

 5.1 Client / Server 职责边界.....14

 5.2 为什么不一开始微服务.....15

 5.3 为什么不一开始 Kubernetes.....15

6. 后端模块设计.....15

7. V1 Scope Control / V1 范围控制.....16

 7.1 P0 必须做.....16

 7.2 P1 可以做.....17

7.3 P2 后续做.....	17
7.4 不进入 V1 的内容.....	17
7.5 避免范围失控的原则.....	17
8. 路线系统设计.....	17
8.1 route_definitions 核心字段.....	18
8.2 路线 JSON 示例.....	18
8.3 SQL schema.....	18
8.4 校验规则和风险.....	19
9. 路线解锁经济系统.....	19
9.1 Trip Prep Fee 公式.....	19
9.2 paywall / grind wall 风险控制.....	19
10. 货币系统与经济闭环.....	20
10.1 sources 与 sinks.....	20
10.2 统一默认数值和公式.....	20
10.3 防通胀策略.....	21
10.4 钱包流水规则.....	21
11. 车辆系统.....	21
11.1 vehicle_definitions 示例.....	21
11.2 当前车辆选择规则.....	21
12. 车辆维护系统.....	22
12.1 为什么燃油不能像体力系统.....	22
13. 在线驾驶系统.....	23
13.1 simulate_online_tick() 伪代码.....	23
14. 离线驾驶系统.....	24
14.1 simulate_offline_progress() 伪代码.....	24
15. Travel Report 离线报告.....	24
15.1 offline_reports 独立表.....	24
15.2 防重复生成规则.....	25
15.3 数据表.....	25
15.4 API 与测试重点.....	25
16. 景点和照片系统.....	26
17. 天气系统.....	26
18. 昼夜系统.....	27
19. Gacha 系统.....	27
20. 每日登录和任务系统.....	28
20.1 Daily Login 奖励和 Day 7 Stamp 风险控制.....	28
20.2 Daily / Weekly Quests.....	29
20.3 period key 与防重复领取.....	29

21. PostgreSQL 数据库设计.....29

21.1 表关系总览.....29

21.x players.....31

21.x wallet_balances.....31

21.x wallet_transactions.....32

21.x player_inventory.....32

21.x vehicle_definitions.....33

21.x player_vehicles.....33

21.x route_definitions.....34

21.x route_segments.....34

21.x landmarks.....35

21.x player_unlocked_routes.....35

21.x player_trips.....35

21.x offline_reports.....36

21.x player_photos.....37

21.x weather_profiles.....37

21.x daily_login_state.....37

21.x quest_definitions.....38

21.x player_quest_progress.....38

21.x gacha_banners.....39

21.x gacha_pool_items.....39

21.x gacha_pity_state.....40

21.x gacha_history.....40

21.x config_versions.....41

21.x analytics_events.....41

21.x suspicious_events.....42

21.x migration_log.....42

21.2 数据库事务边界与一致性要求.....42

22. REST API 设计.....43

22.1 API 总览.....43

22.x GET /player/profile.....45

22.x GET /player/state.....45

22.x POST /player/select-vehicle.....45

22.x GET /routes/available.....46

22.x GET /routes/:route_id.....46

22.x POST /routes/unlock.....46

22.x POST /routes/start.....47

22.x POST /routes/abandon.....47

22.x GET /trip/current.....	47
22.x POST /trip/drive-tick.....	48
22.x POST /trip/claim-offline-report.....	48
22.x POST /trip/complete-landmark.....	48
22.x POST /trip/complete-route.....	49
22.x GET /wallet.....	49
22.x GET /wallet/transactions.....	49
22.x POST /vehicle/refuel.....	50
22.x POST /vehicle/clean.....	50
22.x POST /vehicle/repair.....	50
22.x GET /gacha/banners.....	51
22.x POST /gacha/pull.....	51
22.x GET /gacha/history.....	51
22.x GET /inventory.....	52
22.x POST /inventory/use-item.....	52
22.x GET /daily-login.....	52
22.x POST /daily-login/claim.....	53
22.x GET /quests/daily.....	53
22.x POST /quests/claim.....	53
22.x GET /admin/configs.....	54
22.x POST /admin/configs/draft.....	54
22.x POST /admin/configs/validate.....	54
22.x POST /admin/configs/publish.....	55
22.x POST /admin/configs/rollback.....	55
22.2 后端统一校验规则.....	55
22.3 推荐错误码.....	55
23. Trip Simulation Engine.....	56
23.1 模块职责、输入、输出和依赖.....	56
23.2 核心公式.....	57
23.3 forced stop logic.....	57
23.4 wallet reward logic.....	57
23.5 transaction boundary、concurrency control、idempotency、risk logging.....	58
23.6 工程伪代码.....	58
24. 反作弊和风险控制.....	58
24.1 风险规则.....	59
24.2 suspicious_events 记录规范.....	59
24.3 V1 处理策略.....	59
25. Admin Config System.....	60

25.1 配置发布流程.....	60
25.2 配置校验规则.....	60
25.3 经济模拟器.....	60
25.4 发布前 checklist.....	61
26. Config Version 数据版本控制.....	61
26.1 迁移策略.....	61
26.2 migration preview.....	61
26.3 rollback.....	62
27. Analytics 数据埋点.....	62
27.1 统一事件字段.....	63
27.2 核心指标与判断方法.....	63
28. 技术选型建议.....	64
28.1 为什么不一开始微服务.....	64
28.2 为什么不一开始 Kubernetes.....	64
29. V1 开发阶段路线图.....	64
30. 测试方案.....	65
30.1 关键测试用例示例.....	66
31. 风险清单与解决方案.....	66
32. 最终 Backend Checklist.....	67
32.x 玩家系统.....	67
32.x 钱包系统.....	68
32.x 路线系统.....	68
32.x 教程系统.....	68
32.x 客户端体验层.....	68
32.x 车辆系统.....	68
32.x 维护系统.....	68
32.x 在线驾驶.....	68
32.x 离线驾驶.....	68
32.x Travel Report.....	68
32.x 景点与照片.....	69
32.x 天气与昼夜.....	69
32.x 路线解锁经济.....	69
32.x Gacha.....	69
32.x 每日登录与任务.....	69
32.x Admin Config.....	69
32.x Config Version.....	69
32.x Analytics.....	69
32.x 反作弊.....	69

32.x 测试.....69

32.x 软上线.....70

33. Change Log / 本次优化说明.....70

34. 最终架构师建议.....70

附录 A：默认参数表.....71

附录 B：术语表.....72

附录 C：核心公式索引.....72

0. Executive Summary / 执行摘要

《旅游模拟器 / Travel Simulator》V1 的本质不是赛车游戏、开放世界真实驾驶或现实地图导航系统，而是一套以长途公路旅行感为核心的后端权威旅行模拟系统。玩家选择路线与车辆后，在像素风道路中在线驾驶或离线推进，沿途经历路线段、天气、昼夜、景点停靠、照片卡、车辆维护、路线完成和新路线解锁。

V1 选择 Curated Route Packs / 手工配置路线包，是因为第一版需要控制路线长度、节奏、景点密度、奖励、背景和维护压力。真实全球地图、Google Maps、真实导航、复杂交通 AI、真实天气 API 与大型物理驾驶模拟都会显著扩大工程边界，却不能更快验证核心循环。

Trip Simulation Engine / 旅程模拟引擎是 V1 的技术核心，负责距离、速度、油耗、清洁度、耐久度、强制停靠、路线终点、离线推进和奖励计算。Economy Ledger / 经济流水系统负责所有货币变动的原子化、幂等和可审计。Route Config System / 路线配置系统负责路线包、路段、景点、天气、昼夜、奖励倍率、配置版本和回滚。

后端必须权威。客户端只能提交 Hold to Drive / 长按驾驶、Auto Driving / 自动驾驶、Hold to Boost / 长按加速、领取、拍照、开始路线等玩家意图；距离、奖励、离线时间、抽卡结果、钱包变化都不能由客户端决定。所有关键行为必须由后端埋点，所有数值必须通过 Admin Config 配置。

V1 成功标准是：玩家能顺畅完成第一趟教程，愿意回来领取 Travel Report / 离线旅行报告，愿意维护车辆继续下一段旅程，愿意收集下一张照片卡，并且不会把 Gacha 理解成付费进度墙，也不会把燃油系统理解成体力系统。

核心结论	落地要求
V1 是旅行模拟，不是赛车或真实导航	保持路线包、景点、照片、车辆维护和离线回归的核心体验。
Trip Simulation Engine 优先级最高	先实现后端权威 tick、离线模拟、forced stop、奖励和车辆消耗。
Economy Ledger 必须严谨	所有 Road Coins、Travel Tokens、Souvenir Stamps、Stamp Fragments、Blueprints 变化都写 wallet_transactions。
Route Config System 必须配置化	路线、数值、天气、任务、Gacha、维护价格、奖励倍率均通过后台版本发布。
离线系统服务留存，不替代在线	max_offline_hours=8；离线 Road Coins 为 4/km，低于在线 10/km。
Gacha 只做收藏和外观	Gacha 不影响玩家能否继续旅行，不参与路线解锁。

1. V1 产品与技术定位

1.1 游戏定位

V1 是像素风、长途公路旅行模拟、轻度经营、车辆收集、离线推进、路线解锁与景点照片收集的组合。核心情绪是放松、期待、进度感、沿途发现和回归领取，而不是竞速、开放世界或高精度驾驶。

维度	V1 定位
核心类型	长途旅行模拟 + 轻度经营 + 收藏。
核心动作	选择路线、选择车辆、开始旅程、在线/离线推进、到达景点、

维度	V1 定位
	拍照、维护、解锁路线、收集车辆/皮肤。
关键约束	不做真实全球地图，不接 Google Maps，不做真实导航，不把燃油做成体力，不让 Gacha 卡住核心路线进度。
技术基线	后端权威计算，客户端表现与输入，所有关键数值可配置。

1.2 V1 核心目标

- 验证长途旅行循环：路线选择、行驶推进、景点停靠、拍照奖励、维护、路线完成、路线解锁。
- 搭建后端权威 Trip Simulation Engine，支持在线 tick 和离线模拟。
- 搭建严格 Economy Ledger，所有货币变化都有交易记录。
- 搭建 Route Config System，使策划不改代码也能调整路线、车辆、天气、任务、Gacha 和奖励倍率。
- 实现第一趟旅程教程，让玩家在 5-10 分钟内理解 Hold to Drive、Auto Driving、Hold to Boost、拍照、维护和路线完成。

1.3 V1 非目标

V1 不做真实全球地图、Google Maps、GPS 导航、开放世界自由驾驶、赛车排行、复杂 PvP、多人 MMO、真实车品牌授权、真实付费 Gacha、区块链/NFT、复杂交通 AI、大型物理驾驶模拟、真实天气 API 或 Kubernetes 必需架构。

1.4 V1 成功标准

指标	建议目标	设计含义
Tutorial Completion Rate	>= 75%	第一趟旅程必须短、顺、有正反馈。
First Trip Complete Rate	>= 65%	玩家理解路线完成与奖励。
D1 Retention	>= 30% 软上线观察	离线报告、照片卡和下一条路线形成回归动机。
Offline Report Claim Rate	>= 45%	离线系统真正驱动回归。
First Route Unlock Rate	>= 35%	Souvenir Stamps 不能太难获得。
Economy Inflation	7 日平均 Road Coins 余额不过度上升	维护、升级、出发费和商店消耗要形成健康 sink。

1.5 V1 不做什么

不做真实地图，不做真实导航，不做过度复杂微服务，不做真实付费抽卡，不做真实天气 API，不做复杂交通 AI，不做 MMO，不做区块链，不让燃油成为体力，不让 Gacha 成为路线进度墙。

1.6 技术设计原则

原则	说明	落地方式
后端权威	客户端不决定距离、时间、奖励、抽卡结果。	drive tick、离线模拟、发奖、抽卡全部由后端计算。

原则	说明	落地方式
配置优先	数值不写死，策划可调参。	Admin Config + config_versions + checksum + publish/rollback。
经济可审计	所有货币变化必须有流水。	wallet_balances + wallet_transactions + idempotency_key。
离线克制	离线用于留存，不替代在线体验。	max_offline_hours=8，离线速度和奖励低于在线。
核心进度非付费墙	Gacha 不影响能否继续旅行。	路线解锁使用 Souvenir Stamps，Gacha 使用 Travel Tokens。

2. 核心玩家循环

2.1 主循环

选择路线 -> 选择车辆 -> 支付 Trip Prep Fee -> 开始旅程
-> 在线 Hold to Drive / Auto Driving / Hold to Boost 或离线推进
-> 消耗油量、清洁度、耐久度
-> 到达景点 -> 拍照 -> 得照片
-> 继续行驶 -> 完成路线 -> 获得 Road Coins / Travel Tokens / Souvenir Stamps
-> 维护车辆 -> 解锁新路线 -> 开始下一段旅程

2.2 每日循环

登录 -> 领取 Daily Login 的 Stamp Fragments / Road Coins
-> 查看 Travel Report -> 领取 pending rewards
-> 完成每日任务 -> 维护车辆 -> 推进当前路线或开启新路线

2.3 离线回归循环

玩家离线 -> 后端记录 last_seen_at / last_simulated_at
-> 玩家回归 -> 后端计算 offline_seconds 与 final_offline_distance
-> 生成 Travel Report pending rewards
-> 玩家查看报告 -> claim -> 钱包流水入账 -> 继续旅程

循环	目标	关键系统	V1 风险控制
路线解锁循环	让玩家看到下一段目的地。	Souvenir Stamps、Stamp Fragments、route unlock。	解锁成本 clamp，禁止每次出发重复收 Stamps。
车辆维护循环	让旅行有沉浸式经营感。	Fuel、Cleanliness、Durability。	油量不做体力；清洁度不阻止行驶；耐久只温和影响。
照片收集循环	制造沿途发现和收藏。	Landmark、Photo Card、Album。	首次奖励高，重复奖励低，防止刷资源。
Gacha 收藏循环	提供车辆/皮肤/外观收集。	Travel Tokens、Blueprints、pity。	不影响路线进度，不出售核心推进能力。

3. V1 Client Gameplay Presentation / 主界面表现与玩家体验层

3.1 主屏幕驾驶表现

主屏幕应让玩家持续感到“车正在路上”。玩家车辆以像素风视觉在道路上前进，路边环境根据 route segment 切换。客户端可以根据后端返回的 current_distance_km、segment、weather、day_phase 做动画插值，但最终状态以后端返回为准。

3.2 像素风 route segment 背景切换

路线段决定背景：城市、海岸、沙漠、森林、雪山、平原、夜景、高速公路、山路等。每个 segment 绑定 background_pack_id、terrain_type、biome、speed/fuel/cleanliness/durability multiplier。客户端无需知道真实经纬度，只需根据后端当前 segment 播放对应背景。

3.3 天气/昼夜表现

天气和昼夜影响画面色调、粒子、UI 氛围和照片卡。Sunny 更明亮，Rain 增加雨线和反光，Fog 降低远景可见度，Snow 增加冷色和雪粒，Sandstorm 叠加黄沙色调。Dawn、Day、Sunset、Night 影响天空、路灯、照片氛围和部分任务。

3.4 Route Board

Route Board 是主界面信息中枢，应显示：下一个城市、下一个景点、剩余距离、当前天气、昼夜阶段、当前路线进度、车辆油量/清洁度/耐久度、是否处于 Forced Stop。显示值来自 GET /player/state 或 GET /trip/current，不由客户端自行结算。

3.5 Travel Report UI

Travel Report 应以可视化方式展示昨夜行驶距离、奖励、油耗、车辆损耗、天气变化、是否到达景点、是否因为 LOW_FUEL / LANDMARK_REQUIRED / ROUTE_END 停靠。报告中的奖励为 pending，玩家点击 claim 后，后端写入 wallet_transactions 并返回结果。

3.6 景点停靠与照片卡

到达景点时车辆自动停靠，屏幕展示 avatar + car + scenery 的照片卡构图。玩家拍照后获得 Photo Card，照片质量由清洁度、天气、昼夜、景点稀有度共同影响。强制景点未处理前不允许继续越过该点。

3.7 客户端预测动画与后端权威状态

客户端可以做短时预测动画，以减少操作延迟，但预测只用于表现。每次 drive tick 后必须用后端返回的 current_distance_km、vehicle_state、rewards、forced_stop_reason 纠正。若预测与后端差距过大，客户端应平滑回拉，而不是继续相信本地状态。

客户端可以做	客户端不能做
播放车辆移动、背景、天气、UI 动画。	决定最终距离、奖励、离线时间、抽卡结果。
提交驾驶模式、领取、拍照、开始路线等意图。	提交 Road Coins / Travel Tokens / Stamps 的变化值。
用后端状态做预测插值。	在本地计算并持久化权威旅程进度。

4. 第一趟旅程教程系统

4.1 教程目标

第一趟旅程必须短、顺、有正反馈。教程不是完整系统说明，而是用一条 80-120 km 的标准化短路线，让玩家理解选择路线、Hold to Drive、Auto Driving、Hold to Boost、景点停靠、第一次拍照、完成路线、维护与后续解锁。

设计项	Clean Final V1 方案
起点/目的地	3 个起点 × 3 个目的地，组成 9 个可选第一趟组合；路线表现不同，但教程节奏相同。
距离	Tutorial route = 80-120 km，推荐默认 100 km。
费用	第一趟路线免费，Trip Prep Fee = 0，车辆免费且油量足够完成。
Hold to Drive	玩家必须完成一次持续长按驾驶，理解主动操作。
Auto Driving	完成第一段手动驾驶后解锁，降低操作疲劳。
Hold to Boost	在 Auto Driving 解锁后作为短时加速教学，倍率 1.10。
第一个景点	路线 30%-45% 处强制停靠，完成第一次拍照。
系统解锁	完成第一趟后解锁完整路线、车库、维护、任务、Gacha、相册。

4.2 教程状态机

```
NOT_STARTED
-> ROUTE_SELECTED
-> HOLD_TO_DRIVE_REQUIRED
-> AUTO_DRIVING_UNLOCKED
-> FIRST_LANDMARK_REACHED
-> PHOTO_TAKEN
-> ROUTE_COMPLETED
-> FULL_SYSTEM_UNLOCKED
```

4.3 教程数据字段

```
tutorial_state: enum
selected_start_id: string
selected_destination_id: string
tutorial_route_id: uuid
hold_drive_seconds_done: integer
auto_driving_unlocked_at: timestamptz | null
first_landmark_id: uuid | null
first_photo_id: uuid | null
route_completed_at: timestamptz | null
full_system_unlocked_at: timestamptz | null
version: integer
```

4.4 教程 API

API	用途	关键校验
GET /player/state	返回 tutorial_state、unlocked_features、current_trip。	玩家存在；状态一致；如有 pending offline_report 优先返回。
GET /routes/available	返回教程可用起点/目的地或完整路线。	教程未完成时只返回 Tutorial route。
POST /routes/start	开始教程路线。	Tutorial 免费；无 active trip；车辆可用。
POST /trip/drive-tick	提交教程驾驶意图。	tick 合法；mode 已解锁；后端计算距离。
POST /trip/complete-landmark	完成第一个景点拍照。	玩家已到达 landmark；首次奖励不重复。

API	用途	关键校验
POST /trip/complete-route	完成教程路线并解锁系统。	到达终点；未重复完成。

4.5 教程测试重点

状态转换顺序、重复请求幂等、断线重连、第一趟免费、强制景点停靠、Auto Driving 解锁时机、第一次拍照奖励、完成后完整系统解锁。

4.6 教程风险

教程过慢、第一景点过远、Auto Driving 解锁太晚、Hold to Boost 倍率太高、完成后目标不清。解决方案是把第一趟路线控制在 80-120 km，第一景点放在 30%-45%，Auto 早解锁，并在完成后明确展示下一条可解锁路线。

5. 总体技术架构

V1 推荐“单体后端 + 模块化服务”的架构。早期不拆复杂微服务，因为核心风险是玩法循环、经济平衡、离线模拟和配置运营，而不是服务拆分。单体后端通过清晰模块边界、事务边界和配置版本实现稳定交付；当 DAU、团队规模和功能复杂度增长后再逐步拆分服务。

Client (Unity / Godot)
-> HTTPS / JSON
Backend API (Node.js + NestJS + TypeScript, modular monolith)
-> Trip Simulation Engine / Wallet Service / Route Config System
PostgreSQL + Redis
-> Admin Config / Analytics / Risk Dashboard

层级	职责	V1 推荐
Client	表现、输入、动画、UI、路线背景展示、驾驶手感。	不计算最终距离、奖励、抽卡、离线时长。
Backend API	鉴权、命令处理、校验、事务、聚合状态。	NestJS + TypeScript。
Trip Simulation Engine	在线 tick、离线推进、车辆状态消耗、强制停靠、奖励计算。	后端核心模块，独立 service + unit tests。
PostgreSQL	主存储、钱包、配置版本、路线、trip、车辆、Gacha、任务。	强一致事务与审计。
Redis	限流、幂等短缓存、分布式锁、热点配置缓存。	不作为权威最终数据源。
Admin Panel	配置草稿、校验、发布、回滚、经济模拟、风险查询。	Retool 或自研后台。
Analytics	事件埋点、漏斗、留存、经济监控。	PostHog / Firebase + 本地事件表。

5.1 Client / Server 职责边界

能力	客户端	后端
驾驶输入	提交 hold/auto/boost 意图和本地显示。	校验 tick 时长并计算距离。

能力	客户端	后端
速度/距离	可做预测展示。	最终权威值。
离线时间	只触发请求。	基于 server_now 和 last_simulated_at 计算。
奖励	展示结果。	计算、写流水、返回结果。
抽卡	播放动画。	决定结果、保底、记录历史、扣 Travel Tokens。
配置	读取版本和素材 key。	发布、校验、checksum、版本锁定。

5.2 为什么不一开始微服务

V1 的钱包、旅程、车辆、任务之间事务耦合高，过早拆服务会增加分布式事务复杂度。模块化单体可以保留单库事务与快速迭代能力。

5.3 为什么不一开始 Kubernetes

Kubernetes 会带来部署、监控、安全、成本和运维复杂度。V1 可以先使用 Render、Railway、Supabase、AWS App Runner、GCP Cloud Run 等托管方案，等出现明确瓶颈再迁移。

6. 后端模块设计

每个模块必须明确职责、输入、输出、依赖、优先级和后续扩展。Trip Simulation Engine 不直接暴露 HTTP，由 Trip Service 在事务中调用。Wallet Service 不允许其他模块直接 UPDATE 余额，必须通过 grant/spend 接口生成流水。

模块	职责	P0/P1/P2 优先级	主要依赖
Auth Service	登录、匿名账号、第三方账号绑定、token 校验。	P0 匿名登录与 token；P1 第三方登录；P2 设备风控。	players
Player Service	玩家档案、教程状态、功能解锁、last_seen_at、risk_status。	P0 profile/state；P1 设置；P2 称号地区。	Auth、Trip、Wallet
Route Service	可用路线、路线详情、路线解锁、开始路线前校验。	P0 curated routes；P1 活动路线；P2 路线收藏。	Wallet、Config、Trip
Trip Service	旅程生命周期 start/current/abandon/complete。	P0 active trip；P1 多 trip 历史；P2 多车派遣。	Trip Engine、Vehicle、Wallet
Trip Simulation Engine	在线/离线推进、消耗、停靠、奖励。	P0 核心；P1 事件；P2 突发事件。	Route、Vehicle、Weather、Wallet
Vehicle Service	车辆定义、玩家车辆、选择、升级、皮肤。	P0 车库和选择；P1 升级/皮肤；P2 改装。	Wallet、Inventory
Maintenance Service	加油、洗车、修理价格和执行。	P0 三项维护；P1 区域价格；P2 服务站折扣。	Wallet、Vehicle
Economy / Wallet Service	余额、流水、原子化发奖/扣费、	P0 wallet_transactions；P1 经	所有发奖模块

模块	职责	P0/P1/P2 优先级	主要依赖
	幂等。	济报表；P2 回滚工具。	
Inventory Service	物品、皮肤、Blueprints、消耗品。	P0 基础库存；P1 蓝图兑换；P2 合成系统。	Wallet、Gacha
Landmark / Photo Service	景点到达、拍照、照片卡、相册奖励。	P0 required stop/first photo；P1 album；P2 照片变体。	Trip、Wallet
Weather / DayNight Service	路线天气生成、昼夜计算、效果倍率。	P0 确定性模拟；P1 照片倍率；P2 季节。	Config、Trip Engine
Gacha Service	Banner、抽取、保底、重复转Blueprints、历史。	P1 收藏向 Gacha；P2 付费合规。	Wallet、Inventory
Daily Login Service	7 日循环、period key、领奖。	P1 登录奖励；P2 活动签到。	Wallet
Quest Service	每日/每周任务定义、事件驱动进度、领奖。	P1 每日/每周；P2 成就。	Analytics、Wallet
Admin Config Service	配置草稿、校验、发布、回滚、checksum、审计。	P0 路线/经济配置；P1 模拟器 UI；P2 实验平台。	PostgreSQL、Redis
Analytics Service	事件上报、标准字段、漏斗、经济指标。	P0 核心事件；P1 看板；P2 实验分析。	PostHog/Firebase
Anti-Cheat / Risk Service	风险规则、异常记录、限速、幂等。	P0 标记和限制；P1 风险后台；P2 自动处置。	Trip、Wallet、Gacha

7. V1 Scope Control / V1 范围控制

V1 的目标是证明核心循环，而不是做“内容最多、技术最复杂”的版本。所有功能必须按 P0/P1/P2 管控，避免真实地图、多人、复杂商业化和过度后端架构拖垮第一版。

7.1 P0 必须做

P0 模块	必须交付
玩家账号和存档	匿名/基础账号、玩家 profile、教程状态、last_seen_at。
第一趟教程	3×3 起终点、80-120 km、免费路线、Hold to Drive、Auto Driving、第一个景点、第一次拍照。
路线包系统	Curated Route Packs、route_definitions、segments、landmarks、config_version。
Trip Simulation Engine	在线 tick、离线推进、forced stop、奖励、油耗、车辆消耗。
在线驾驶与 Auto Driving	Hold to Drive=1.00、Auto Driving=0.85、基础 tick 限流与幂等。
离线推进和 Travel Report	max_offline_hours=8、pending report、claim 后入账、防重复生成。
货币与钱包流水	Road Coins / Travel Tokens / Souvenir Stamps / Stamp Fragments / Blueprints，所有变化进 wallet_transactions。

P0 模块	必须交付
车辆维护	油量、清洁度、耐久度，加油、洗车、修理。
景点强制停靠和照片卡	required_stop、photo card、first photo reward。
路线解锁	Souvenir Stamps 永久解锁，Trip Prep Fee 使用 Road Coins。
基础任务和 Admin Config	基础每日任务、配置草稿/校验/发布/回滚。
核心 Analytics 与反作弊	核心事件、tick limit、speed cap、suspicious_events。

7.2 P1 可以做

Gacha、Blueprints、更完整的相册、每周任务、外观商店、车辆升级、更完整后台配置 UI、风险后台。

7.3 P2 后续做

真实天气 API、真实时区、多人车队、复杂突发事件、复杂交通 AI、全球真实路线、活动路线包、社交分享、更多照片变体。

7.4 不进入 V1 的内容

开放世界真实地图、Google Maps、真实导航、多人大世界、复杂 PvP、真实车品牌授权、真实付费 Gacha、区块链/NFT、Kubernetes 必需项、真实天气 API 必需项、复杂交通 AI、大型物理驾驶模拟。

7.5 避免范围失控的原则

任何功能若不能增强第一趟旅程、路线推进、离线回归、照片收集、车辆维护、路线解锁或经济可审计，应推迟到 P1/P2。每个新增系统必须能回答：是否服务 V1 成功标准，是否可后台配置，是否能埋点验证，是否会破坏后端权威。

8. 路线系统设计

V1 使用 Curated Route Packs / 手工配置路线包。每条路线不是现实导航数据，而是策划配置的 route_definition、route_segments、landmarks、rest_points、weather_profile、day_night_profile、background_pack 和 reward 参数。

路线类型	距离范围	V1 用途	节奏建议
Tutorial	80-120 km	第一趟教学	1 个必停景点，免费。
Short	120-300 km	早期日常路线	1-2 个景点，1 天可完成。
Medium	300-800 km	核心中期路线	2-4 个景点，支持离线推进。
Long	800-1500 km	长期目标	4-6 个景点，维护与补给更重要。
Epic	1500-3000 km	大型路线包	6-10 个景点，V1 后段或活动内容。

8.1 route_definitions 核心字段

字段	说明
route_id / route_key	全局唯一 ID 和人类可读 key。
config_version_id	配置版本；旅程开始时锁定。
name / region	展示名和区域。
start_node / destination_node	起点和目的地配置 ID。
total_distance_km	路线总距离。
route_type / difficulty	Tutorial / Short / Medium / Long / Epic；难度 1-5。
unlock_cost_stamps	永久解锁 Souvenir Stamps 成本。
trip_prep_fee_coins	每次出发 Road Coins 准备费，必须封顶。
weather_profile_id / day_night_profile	天气权重与昼夜配置。
background_pack_id	像素背景素材包。

8.2 路线 JSON 示例

```
{
  "route_id": "rt_tutorial_coast_001",
  "route_config_version": 12,
  "name": "海湾小镇到灯塔公路",
  "route_type": "Tutorial",
  "start_node": "start_bay_town",
  "destination_node": "dest_lighthouse",
  "total_distance_km": 102,
  "difficulty": 1,
  "unlock_cost_stamps": 0,
  "trip_prep_fee_coins": 0,
  "reward_multiplier": 1.0,
  "weather_profile_id": "wp_coast_easy",
  "day_night_profile_id": "dnp_default_day",
  "background_pack_id": "bg_coast_pixel_v1",
  "segments": [
    { "index": 0, "start_km": 0, "end_km": 34, "terrain": "coast", "speed_multiplier": 1.0 },
    { "index": 1, "start_km": 34, "end_km": 70, "terrain": "forest", "speed_multiplier": 0.92 },
    { "index": 2, "start_km": 70, "end_km": 102, "terrain": "highway", "speed_multiplier": 1.08 }
  ],
  "landmarks": [
    { "landmark_id": "lm_first_lighthouse", "distance_km": 42, "required_stop": true,
      "rarity": "Common", "base_photo_coins": 80 }
  ]
}
```

8.3 SQL schema

```
CREATE TABLE route_definitions (
  route_id UUID PRIMARY KEY,
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),
  route_key TEXT NOT NULL,
  name TEXT NOT NULL,
  region TEXT NOT NULL,
  start_node TEXT NOT NULL,
  destination_node TEXT NOT NULL,
  route_type TEXT NOT NULL CHECK (route_type IN ('Tutorial','Short','Medium','Long','Epic')),
  total_distance_km NUMERIC(10,2) NOT NULL CHECK (total_distance_km > 0),
  difficulty INT NOT NULL CHECK (difficulty BETWEEN 1 AND 5),
  unlock_cost_stamps INT NOT NULL DEFAULT 0,
  trip_prep_fee_coins INT NOT NULL DEFAULT 0,
  reward_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  weather_profile_id UUID REFERENCES weather_profiles(weather_profile_id),
  day_night_profile JSONB NOT NULL DEFAULT '{}::jsonb',
  background_pack_id TEXT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  is_deprecated BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
UNIQUE(config_version_id, route_key)
);

CREATE TABLE route_segments (
  segment_id UUID PRIMARY KEY,
  route_id UUID NOT NULL REFERENCES route_definitions(route_id) ON DELETE CASCADE,
  segment_index INT NOT NULL,
  start_km NUMERIC(10,2) NOT NULL,
  end_km NUMERIC(10,2) NOT NULL,
  terrain_type TEXT NOT NULL,
  speed_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  fuel_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  cleanliness_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  durability_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,
  metadata JSONB NOT NULL DEFAULT '{} '::jsonb,
  UNIQUE(route_id, segment_index),
  CHECK (end_km > start_km)
);

CREATE INDEX idx_route_segments_route_range ON route_segments(route_id, start_km, end_km);
```

8.4 校验规则和风险

segments 必须从 0 连续覆盖到 total_distance_km，不得重叠或断档；landmark.distance_km 必须在路线范围内；Tutorial 距离必须 80-120 km；新 trip 使用 live config，进行中的 trip 使用开始时 config_version；deprecated 路线不允许新开但可以让历史 trip 完成。

9. 路线解锁经济系统

Route Unlock Economy 的目标是制造中长期目标，但不能变成 paywall 或 grind wall。V1 采用 Souvenir Stamps 永久解锁路线，Stamp Fragments 作为轻量日常进度。核心 Souvenir Stamps 主要来自路线完成、相册完成和每周任务，而不是签到。

```
unlock_cost = round(base_cost + total_distance_km / 350 + landmark_count * 0.4 + difficulty_modifier)
route_type clamp:
Tutorial = 0
Short = 1-2
Medium = 3-5
Long = 6-9
Epic = 12-18
```

规则	设计理由
Tutorial 免费	第一趟必须顺畅，不能在未理解玩法前遇到货币门槛。
Stamps 永久解锁路线	让路线成为明确目标，解锁后有拥有感。
解锁后不重复收 Stamps	避免每次出发都被收核心进度货币。
Trip Prep Fee 只收 Road Coins 且封顶	模拟旅行准备成本，但不阻止游玩。
不用 Travel Tokens 解锁路线	避免 Gacha 货币与核心进度绑定。

9.1 Trip Prep Fee 公式

```
trip_prep_fee = clamp(round(route.total_distance_km * 0.35), 20, 300)
Tutorial route: trip_prep_fee = 0
已解锁路线：每次出发只扣 Road Coins，不扣 Stamps
未解锁路线：先完成永久 unlock，再扣 Trip Prep Fee 开始旅程
```

9.2 paywall / grind wall 风险控制

路线解锁不使用 Travel Tokens，不依赖 Gacha；Short / Medium 路线成本 clamp，确保早期能通过路线完成、相册和周任务获得足够 Stamps；每日登录以 Stamp Fragments 为主，不能让纯签到跳过旅行内容。

10. 货币系统与经济闭环

经济系统核心原则是：核心推进与收藏货币分离，所有货币变化有交易流水，sources 与 sinks 可度量、可调参、可回滚。

货币	定位	主要来源	主要消耗	风险控制
Road Coins	基础软货币，维护、Trip Prep Fee、升级和日常商店。	在线驾驶 10/km、离线驾驶 4/km、拍照、任务、路线完成。	加油、洗车、修理、Trip Prep Fee、升级。	防通胀，看平均余额和收入/消耗比。
Travel Tokens	收藏/Gacha 货币。	在线每 10 km 1 个、离线每 20 km 1 个、任务、少量路线奖励。	Gacha 单抽/十连、少量外观。	不能用于路线解锁。
Souvenir Stamps	路线永久解锁货币。	路线完成、相册完成、每周任务。	永久解锁路线。	不能直接大量签到产出。
Stamp Fragments	弱进度碎片。	每日登录、少量每日任务。	10 个 = 1 Souvenir Stamp。	防止纯签到跳过核心玩法。
Blueprints	重复 Gacha 补偿与兑换材料。	重复物品转化、活动任务。	车辆碎片兑换、皮肤兑换、升级材料。	补偿过高会冲击稀有度。

10.1 sources 与 sinks

类别	项目	货币	约束
Source	在线驾驶	Road Coins / Travel Tokens	在线 1 km = 10 Road Coins；在线 10 km = 1 Travel Token。
Source	离线驾驶	Road Coins / Travel Tokens	离线 1 km = 4 Road Coins；离线 20 km = 1 Travel Token。
Source	拍照	Road Coins / Photo Card	首次奖励高，重复奖励低。
Source	路线完成	Coins / Tokens / Stamps	按路线类型和难度。
Sink	Trip Prep Fee	Road Coins	$\text{clamp}(\text{round}(\text{distance} \times 0.35), 20, 300)$ ，Tutorial=0。
Sink	维护	Road Coins	加油、洗车、修理成本必须可承受。
Sink	Gacha	Travel Tokens	单抽 20，十连 180。
Sink	路线解锁	Souvenir Stamps	永久解锁，不重复扣。

10.2 统一默认数值和公式

```
online_road_coins = floor(online_distance_km * 10 * route_reward_multiplier * active_event_multiplier)
offline_road_coins = floor(offline_distance_km * 4 * route_reward_multiplier)

online_token_meter += online_distance_km
while online_token_meter >= 10:
    grant 1 Travel Token
    online_token_meter -= 10

offline_token_meter += offline_distance_km
while offline_token_meter >= 20:
    pending 1 Travel Token
    offline_token_meter -= 20

10 Stamp Fragments = 1 Souvenir Stamp
```

```
trip_prep_fee = clamp(round(route.total_distance_km * 0.35), 20, 300)
upgrade_cost = round(base_upgrade_cost * rarity_multiplier * level ^ 1.7)
```

10.3 防通胀策略

维护、Trip Prep Fee、车辆升级和外观商店是主要 sink；在线/离线奖励必须分层，离线奖励低于在线；路线完成和相册可以给 Stamps，但必须通过 clamp 控制解锁速度。所有调参必须通过 Admin Config + 经济模拟器验证。

10.4 钱包流水规则

wallet_balances 只保存当前余额；wallet_transactions 保存所有增减流水。任何发奖/扣费必须在数据库事务中完成，必须有 reason、source_type、source_id、idempotency_key。禁止客户端提交 reward amount。

11. 车辆系统

车辆系统服务于旅行推进、维护、收藏和外观。V1 不应把车辆稀有度做成核心路线通行证，普通车辆必须能完成所有基础路线；稀有车辆可以更好看、更省油、更适合离线，但不能决定能否继续旅行。

属性	说明
base_speed_kmph	基础速度，用于在线距离计算。
fuel_capacity / fuel_consumption_per_km	油箱容量与每公里油耗。
durability / durability_loss_per_km	最大耐久度与每公里耐久损耗。
cleanliness / cleanliness_loss_per_km	最大清洁度与每公里清洁损耗。
offline_efficiency	离线效率，影响离线速度上限。
weather_resistance	天气抗性 0-1。
rarity	Common / Rare / Epic / Legendary。
skin_id / upgrade_level	当前皮肤与升级等级。

11.1 vehicle_definitions 示例

```
{
  "vehicle_def_id": "veh_van_common_001",
  "display_name": "蓝色旅行小面包",
  "rarity": "Common",
  "base_speed_kmph": 72,
  "fuel_capacity": 45,
  "fuel_consumption_per_km": 0.075,
  "durability": 100,
  "durability_loss_per_km": 0.018,
  "cleanliness": 100,
  "cleanliness_loss_per_km": 0.035,
  "offline_efficiency": 0.60,
  "weather_resistance": 0.15,
  "default_skin_id": "skin_van_blue_default",
  "upgrade_curve_key": "default_v1"
}
```

11.2 当前车辆选择规则

玩家同一时间只能选择一辆 active vehicle；V1 默认不允许旅程中换车，避免车辆状态与路线模拟复杂化；可在 P2 允许 rest_point 停靠换车。开始旅程时锁定 player_vehicle_id；油量为 0 不能继续推进；清洁度和耐久低不直接阻止行驶，只影响表现、照片、油耗、离线效率和少量速度。

12. 车辆维护系统

维护系统包括 Fuel、Cleanliness、Durability。它必须增强旅行沉浸感和轻经营感，但不能变成体力系统。因此油量为 0 可以阻止继续行驶；清洁度低不阻止行驶，只影响照片奖励和外观；耐久低不强制阻止，但温和影响油耗、离线效率和少量速度。

```
fuel_used = distance_km * vehicle.fuel_consumption_per_km * weather_fuel_multiplier * terrain_fuel_multiplier *
durability_fuel_penalty * mode_fuel_multiplier
cleanliness_loss = distance_km * vehicle.cleanliness_loss_per_km * weather_cleanliness_multiplier *
terrain_cleanliness_multiplier
durability_loss = distance_km * vehicle.durability_loss_per_km * terrain_durability_multiplier *
adjusted_weather_durability_multiplier
adjusted_weather_durability_multiplier = 1 + (weather_durability_multiplier - 1) * (1 -
vehicle.weather_resistance)

refuel_cost = ceil(missing_fuel * fuel_price_per_liter * region_price_multiplier)
clean_cost = ceil(base_clean_cost + clean_points * clean_price_per_point * rarity_cost_multiplier)
repair_cost = ceil(base_repair_cost + repair_points * repair_price_per_point * rarity_cost_multiplier)
```

项目	建议默认	说明
fuel_price_per_liter	2 Road Coins	早期加油成本可感知但不压迫。
base_clean_cost	15 Road Coins	洗车基础费。
clean_price_per_point	0.8 Road Coins	从 50 洗到 100 约 55 Coins。
base_repair_cost	25 Road Coins	修理基础费。
repair_price_per_point	1.2 Road Coins	修理比洗车贵，但不应过高。
durability_speed_penalty_min	0.85	耐久很低时速度最低不低于 85%。
durability_fuel_penalty_max	1.20	耐久很低时油耗最高 +20%。

天气/地形	speed	fuel	cleanliness	durability	photo
Sunny	1.00	1.00	1.00	1.00	1.05
Cloudy	0.98	1.00	1.02	1.00	1.00
Rain	0.92	1.08	1.40	1.08	0.95
Fog	0.88	1.04	1.08	1.03	1.10
Snow	0.80	1.15	1.25	1.18	1.08
Sandstorm	0.78	1.18	1.60	1.25	0.90
highway terrain	1.08	0.95	0.95	0.95	-
mountain terrain	0.82	1.20	1.15	1.25	-
desert terrain	0.90	1.12	1.45	1.10	-

12.1 为什么燃油不能像体力系统

燃油是沉浸式维护系统，不是商业闸门。V1 不应设计等待恢复油量，也不应让玩家因油量长期无法游玩。加油消耗 Road Coins，Road Coins 可通过驾驶、拍照、任务和路线完成获得；第一趟教程必须保证油量足够。

13. 在线驾驶系统

在线驾驶通过 server-authoritative tick 实现。客户端提交玩家意图：Hold to Drive、Auto Driving 或 Hold to Boost；后端根据服务端时间、tick 上限、车辆状态、路线段、天气和模式倍率计算距离与消耗。

```
distance_gain = vehicle_speed_kmph * duration_seconds / 3600 * mode_multiplier * condition_multiplier *
segment_speed_multiplier
condition_multiplier = durability_speed_multiplier * weather_speed_multiplier * fuel_state_multiplier
```

模式	倍率	最终推荐理由
Hold to Drive	1.00	标准在线收益和速度；玩家主动参与的基础。
Auto Driving	0.85	推荐 0.85，而不是 0.75 或 0.90。0.75 太慢会像惩罚；0.90 太接近手动会削弱 Hold to Drive 意义。
Hold to Boost	1.10	推荐 1.10，而不是 1.25。1.25 容易导致速度异常、奖励膨胀和路线节奏失控；1.10 足够有反馈。

规则	建议值/说明
tick_duration	客户端建议每 5-10 秒提交一次；后端按 server_now - last_simulated_at 计算。
max_online_tick_seconds	15 秒；超过部分不计算或按上限计算并记录风险。
每分钟有效驾驶时间	最多 60 秒；重复/并发 tick 通过锁和 idempotency 控制。
fuel check	distance_gain 受 fuel_limited_distance clamp。
forced stop	到达 required_landmark、route_end、low_fuel 等必须停靠。
online reward	Road Coins 即时入账，Travel Tokens 使用 meter 累积。

13.1 simulate_online_tick() 伪代码

```
function simulate_online_tick(player_id, trip_id, client_mode, idempotency_key):
    assert client_mode in ['HOLD_TO_DRIVE', 'AUTO_DRIVING', 'HOLD_TO_BOOST']
    begin transaction
        if exists idempotency_key:
            return previous_result
        trip = SELECT * FROM player_trips WHERE trip_id = ? FOR UPDATE
        assert trip.player_id == player_id and trip.status == 'ACTIVE'
        vehicle = SELECT * FROM player_vehicles WHERE id = trip.player_vehicle_id FOR UPDATE
        now = server_now()
        duration = clamp(seconds_between(now, trip.last_simulated_at), 0, max_online_tick_seconds)
        segment = route.segment_at(trip.current_distance_km)
        weather = weather_service.get_weather(player_id, trip, trip.current_distance_km)
        distance_raw = calculate_distance_gain(vehicle, duration, client_mode, segment, weather)
        fuel_limited_distance = vehicle.current_fuel / calculate_effective_fuel_per_km(vehicle, segment, weather,
client_mode)
        stop = check_for_forced_stop(trip, route, distance_raw, fuel_limited_distance)
        final_distance = min(distance_raw, stop.distance_limit)
        apply fuel / cleanliness / durability losses
        update trip distance and last_simulated_at
        rewards = grant_trip_rewards(player_id, trip, final_distance, mode='ONLINE')
        emit analytics and risk logs
    commit
    return simulation_result(trip, vehicle, rewards, stop)
```

14. 离线驾驶系统

离线驾驶由后端计算，客户端时间完全不可信。玩家回归时，后端根据 server_now、player.last_seen_at 与 trip.last_simulated_at 计算离线秒数，再按车辆、路线段、天气、油量和强制停靠点确定最终离线距离。

```
offline_seconds = min(server_now - max(player.last_seen_at, trip.last_simulated_at), max_offline_hours * 3600)
max_offline_hours = 8
base_offline_speed_kmph = 30

offline_speed_kmph = min(base_offline_speed_kmph, vehicle.base_speed_kmph * vehicle.offline_efficiency)
    * durability_offline_multiplier
    * weather_speed_multiplier
    * segment_speed_multiplier

raw_offline_distance = offline_speed_kmph * offline_seconds / 3600
final_offline_distance = min(raw_offline_distance, fuel_limited_distance, distance_to_next_required_landmark,
distance_to_route_end, distance_to_forced_stop)
```

停靠原因	说明	结果
LANDMARK_REQUIRED	到达 required_stop 景点。	生成报告，trip.status=FORCED_STOP，需玩家拍照/确认继续。
ROUTE_END	到达路线终点。	生成报告，可进入 complete-route。
LOW_FUEL	油量不足。	trip.status=FORCED_STOP，需加油后继续。
MAX_OFFLINE_REACHED	达到 max_offline_hours。	正常报告，未强制停。
CONFIG_OR_ROUTE_ERROR	配置异常。	停止模拟，记录 suspicious/config issue。

14.1 simulate_offline_progress() 伪代码

```
function simulate_offline_progress(player_id, trip_id):
    begin transaction
    player = SELECT * FROM players WHERE player_id = ? FOR UPDATE
    trip = SELECT active trip FOR UPDATE
    if trip is null: update player.last_seen_at; commit; return no_report
    if exists unclaimed offline_report for trip:
        return existing_pending_report
    now = server_now()
    offline_seconds = min(seconds_between(now, max(player.last_seen_at, trip.last_simulated_at)), max_offline_hours
* 3600)
    if offline_seconds < min_offline_report_seconds:
        update player.last_seen_at; commit; return no_report
    vehicle = SELECT player_vehicle FOR UPDATE
    simulate by route segment and weather bucket
    clamp by fuel_limited_distance, required_landmark, route_end
    update trip and vehicle
    insert offline_reports with claimed=false and pending rewards
    update player.last_seen_at and trip.last_simulated_at
    commit
    return report
```

15. Travel Report 离线报告

Travel Report 是离线系统的回归奖励界面和审计载体。报告中的奖励先 pending，玩家点击领取后才写入钱包流水，避免“看不到但已发奖”的体验问题，并通过 report_id、claimed 和 idempotency_key 防止重复领取。

15.1 offline_reports 独立表

独立 offline_reports 表是推荐方案，因为更利于审计、幂等、历史展示、客服查询、Analytics 统计，以及未来多车/多报告扩展。不要只把 offline_report_pending 塞进 player_trips JSON。

15.2 防重复生成规则

关键规则：如果当前 trip 已经存在未领取的 offline_report，GET /player/state 不应该再次生成新的 offline_report。系统应优先返回已有 pending report。只有当前 report 被领取后，下一次有效离线周期才可以生成新的 report。

规则落地：

```
function get_or_generate_offline_report(player_id, trip_id):
  begin transaction
  trip = SELECT * FROM player_trips WHERE trip_id=? FOR UPDATE
  existing = SELECT * FROM offline_reports
    WHERE player_id=? AND trip_id=? AND claimed=false
    ORDER BY generated_at DESC LIMIT 1 FOR UPDATE
  if existing:
    return existing
  if not enough_offline_seconds:
    return null
  return simulate_offline_progress_and_insert_report()
```

15.3 数据表

```
CREATE TABLE offline_reports (
  report_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id UUID NOT NULL REFERENCES players(player_id),
  trip_id UUID NOT NULL REFERENCES player_trips(trip_id),
  generated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  offline_seconds BIGINT NOT NULL,
  distance_travelled_km NUMERIC(12,3) NOT NULL,
  road_coins_pending INT NOT NULL DEFAULT 0,
  travel_tokens_pending INT NOT NULL DEFAULT 0,
  fuel_used NUMERIC(10,3) NOT NULL DEFAULT 0,
  cleanliness_loss NUMERIC(10,3) NOT NULL DEFAULT 0,
  durability_loss NUMERIC(10,3) NOT NULL DEFAULT 0,
  weather_summary JSONB NOT NULL DEFAULT '{}'::jsonb,
  landmark_reached JSONB,
  forced_stop_reason TEXT,
  claimed BOOLEAN NOT NULL DEFAULT FALSE,
  claimed_at TIMESTAMPTZ,
  claim_idempotency_key TEXT
);
CREATE INDEX idx_offline_reports_player_claimed ON offline_reports(player_id, claimed, generated_at DESC);
CREATE UNIQUE INDEX uq_offline_claim_key ON offline_reports(player_id, claim_idempotency_key) WHERE
claim_idempotency_key IS NOT NULL;
CREATE UNIQUE INDEX uq_one_pending_report_per_trip ON offline_reports(player_id, trip_id) WHERE claimed = FALSE;
```

15.4 API 与测试重点

API	用途	后端校验
GET /player/state	返回 current_trip 和未领取 offline_report。	如果已有 pending report，返回已有 report，不生成新的。
GET /trip/current	返回当前旅程和 pending report。	仅当前玩家；trip 状态一致。
POST /trip/claim-offline-report	领取 pending rewards。	SELECT report FOR UPDATE; claimed=false; 写 wallet_transactions; 设置 claimed=true。

测试重点：重复打开 /player/state 不产生多个 report；claim 重试不重复发奖；report 已领取后才允许下一次有效离线周期生成新 report；LOW_FUEL、LANDMARK_REQUIRED、ROUTE_END 等停靠原因正确展示。

16. 景点和照片系统

景点系统让路线不只是距离数字，而是沿途发现。照片系统将车辆清洁度、天气、昼夜和景点稀有度连接成轻度策略，但不能让玩家靠重复拍同一个景点刷资源。

数据结构	关键字段
landmark	landmark_id、route_id、distance_km、name、rarity、required_stop、base_photo_coins、album_group、photo_card_key。
photo_card	photo_id、player_id、landmark_id、trip_id、quality_score、weather、day_phase、is_first_capture、reward_transaction_id。
album	album_group、required_photo_keys、completion_reward_stamps、completion_reward_coins、completed_at。

```
photo_quality_score = clamp(base_quality + cleanliness_score + weather_score + day_phase_score +
landmark_rarity_score, 0, 100)
cleanliness_photo_multiplier = 0.8 + cleanliness / 100 * 0.4
photo_coins = floor(landmark.base_photo_coins * cleanliness_photo_multiplier * weather_photo_multiplier *
first_photo_multiplier * route_reward_multiplier)
```

奖励类型	设计
首次拍照奖励	照片卡 + Road Coins + 相册进度；稀有景点可给少量 Travel Tokens。
重复拍照奖励	低额 Road Coins 或无货币，只更新更高质量照片，防止刷资源。
相册完成奖励	Souvenir Stamps + Road Coins + 展示徽章。
清洁度影响	清洁度越高照片奖励越高，但最低 0.8 倍，不应过度惩罚。
天气/昼夜影响	不同天气和昼夜可影响质量与图鉴变体。

```
function complete_landmark_photo(player_id, trip_id, landmark_id):
  begin transaction
    trip = SELECT trip FOR UPDATE
    assert trip.current_distance_km >= landmark.distance_km
    assert trip.status == 'FORCED_STOP' and trip.forced_stop_reason == 'LANDMARK_REQUIRED'
    existing = find_player_photos(player_id, landmark_id)
    is_first = existing is null
    score = calculate_photo_quality(...)
    rewards = calculate_photo_reward(is_first, score)
    if is_first: grant first rewards
    else: grant only repeat reward with cap or no reward
    update best_photo if score higher
    mark landmark completed for trip
    trip.status = 'ACTIVE'
  commit
```

17. 天气系统

V1 不接真实天气 API。真实天气会引入地区映射、时区、外部依赖、天气异常、成本和不稳定性，而且不服务 V1 的路线包体验。V1 使用 route-based simulated weather，通过固定 seed 和距离 bucket 保证在线/离线可复现。

天气	speed	fuel	cleanliness	durability	photo	体验定位
Sunny	1.00	1.00	1.00	1.00	1.05	标准舒适天气。
Cloudy	0.98	1.00	1.02	1.00	1.00	轻微变化。
Rain	0.92	1.08	1.40	1.08	0.95	增加维护压力。
Fog	0.88	1.04	1.08	1.03	1.10	慢速但照片有氛围。
Snow	0.80	1.15	1.25	1.18	1.08	高难路线天气。
Sandstorm	0.78	1.18	1.60	1.25	0.90	沙漠/困难路线特色。

```
weather_seed = hash(player_id + route_id + trip_id + route_config_version + weather_profile.seed_salt)
weather_bucket = floor(current_distance_km / weather_profile.refresh_distance_km)
weather = weighted_random(seed = hash(weather_seed + weather_bucket), weights = weather_profile.weights)
```

离线模拟复现天气时不能按真实时间随机，而是按 trip、route_config_version 和 current_distance_km 的 bucket 计算。这样在线 tick 与离线推进跨同一距离段会得到相同天气。

18. 昼夜系统

昼夜系统主要服务背景、照片、UI 和沉浸感。V1 不需要真实时区，不需要和现实时间绑定。每条路线配置 start_game_minutes，trip 根据真实经过秒数乘以 game_time_speed_multiplier 计算游戏时间。

```
game_time_speed_multiplier = 10
game_minutes_elapsed = trip.elapsed_real_seconds * game_time_speed_multiplier / 60
current_game_minutes = (route.start_game_minutes + game_minutes_elapsed) % 1440
```

阶段	时间范围	影响
Dawn	05:00-07:00	背景暖色、照片氛围加分、可绑定黎明拍照任务。
Day	07:00-17:00	默认背景，照片稳定。
Sunset	17:00-19:00	照片氛围加分，UI 显示夕阳。
Night	19:00-05:00	夜间背景、路灯、特殊照片卡。

19. Gacha 系统

Gacha 在 V1 的定位是收藏与外观，不影响玩家能否继续旅行。抽卡结果必须由后端生成，概率、池子和保底必须配置化并写入历史。未来如果接真实付费，需要额外合规、概率公示、未成年人保护、退款和地区法规处理，V1 不做真实付费 Gacha。

稀有度	概率
Common	70%
Rare	22%
Epic	7%

稀有度	概率
Legendary	1%

规则	值
单抽价格	20 Travel Tokens
十连价格	180 Travel Tokens
10 抽保底	至少 Rare
30 抽保底	至少 Epic
80 抽保底	至少 Legendary
重复物品	转 Blueprints，按稀有度给不同数量。

```
function pull_gacha(player_id, banner_id, pull_count, idempotency_key):
    assert pull_count in [1, 10]
    begin transaction
    if exists idempotency_key: return previous_pull_result
    banner = select active banner
    cost = 20 if pull_count == 1 else 180
    wallet.spend(player_id, 'TRAVEL_TOKENS', cost, source='GACHA_PULL')
    pity = SELECT gacha_pity_state FOR UPDATE
    for i in 1..pull_count:
        rarity = roll_rarity_with_pity(pity, banner.rules)
        item = weighted_random(pool_items where rarity=rarity)
        if player_already_has(item): grant Blueprints by duplicate rule
        else: grant item to inventory / vehicle collection
        update pity counters
        insert gacha_history
    commit
    return results
```

后端校验：banner active、pool weights > 0、玩家 Tokens 足够、pull_count 只能 1/10。客户端不提交随机种子或结果；服务端 RNG；gacha_history 可审计。

20. 每日登录和任务系统

每日登录和任务系统用于提供轻量目标，不能替代路线完成和照片收集。所有任务进度应事件驱动，例如 DRIVE_DISTANCE、PHOTO_TAKEN、VEHICLE_REFUELED，而不是客户端直接提交“我完成了任务”。

20.1 Daily Login 奖励和 Day 7 Stamp 风险控制

Daily Login 主要给 Stamp Fragments、Road Coins 或少量 Travel Tokens。Day 7 可以给 1 个 Souvenir Stamp，但必须限制为每周最多 1 个，并在运营策略上明确：核心 Souvenir Stamps 仍主要来自路线完成、相册完成和每周任务，避免玩家只靠签到解锁路线。

天数	建议奖励
Day 1	5 Stamp Fragments + 100 Road Coins
Day 2	5 Stamp Fragments + 1 Travel Token
Day 3	8 Stamp Fragments + 150 Road Coins
Day 4	5 Stamp Fragments + 洗车券或 Road Coins

天数	建议奖励
Day 5	8 Stamp Fragments + 2 Travel Tokens
Day 6	10 Stamp Fragments
Day 7	1 Souvenir Stamp + 200 Road Coins；每周最多 1 个完整 Souvenir Stamp。

20.2 Daily / Weekly Ques

任务类型	例子	事件	奖励
Daily	在线驾驶 50 km	DRIVE_DISTANCE_ONLINE	Road Coins + Fragments
Daily	领取离线报告	OFFLINE_REPORT_CLAIMED	Road Coins
Daily	加油一次	VEHICLE_REFUELED	Road Coins
Daily	拍一张景点照片	PHOTO_TAKEN	Fragments / Coins
Weekly	本周行驶 1000 km	DRIVE_DISTANCE_TOTAL	Souvenir Stamp + Coins
Weekly	拍摄 5 张景点照片	PHOTO_TAKEN	Souvenir Stamp / Tokens
Weekly	完成一条路线	ROUTE_COMPLETED	Souvenir Stamp

20.3 period key 与防重复领取

V1 建议统一运营时区，例如 UTC 或 UTC+8。daily_period_key = yyyy-mm-dd@timezone，weekly_period_key = ISO week@timezone。领奖接口必须在事务中锁定 daily_login_state 或 player_quest_progress，claimed_at 非空则返回幂等结果，不重复发奖。

21. PostgreSQL 数据库设计

本章给出 V1 推荐的 PostgreSQL schema。配置表与玩家运行态表分离；玩家、钱包、旅程、离线报告、抽卡历史和风险事件必须可追溯。所有可能产生奖励或扣费的行为都必须落到 wallet_transactions 或对应历史表。

21.1 表关系总览

表名	用途	主键	主要外键	索引建议
players	玩家账号、教程状态、风险状态和 last_seen_at。	player_id	-	auth_provider+external_id, risk_status, last_seen_at
wallet_balances	货币余额快照。	player_id + currency	players	player_id, currency
wallet_transactions	不可变钱包流水，所有经济变化必须进入。	transaction_id	players	player_id+created_at, source_type+source_id, idempotency_key
player_inventory	通用库存、道具、蓝图分类库存。	player_id + item_type + item_id	players	player_id+item_type
vehicle_definitions	车辆配置定义。	vehicle_def_id	config_versions	config_version, rarity,

表名	用途	主键	主要外键	索引建议
				live_status
player_vehicles	玩家拥有车辆与运行状态。	player_vehicle_id	players, vehicle_definitions	player_id, selected unique
route_definitions	路线定义与解锁/奖励配置。	route_id	config_versions, weather_profiles	route_type+is_active, config_version_id
route_segments	路线分段与地形/背景/倍率。	segment_id	route_definitions	route_id+start_km+end_km
landmarks	路线景点、强制停靠与照片卡配置。	landmark_id	route_definitions	route_id+distance_km, album_group_id
player_unlocked_routes	玩家已永久解锁路线。	player_id + route_id	players, route_definitions	player_id, route_id
player_trips	当前/历史旅程主状态。	trip_id	players, route_definitions, player_vehicles	player_id+status, route_id, last_simulated_at
offline_reports	离线报告 pending 奖励与历史审计。	report_id	players, player_trips	player_id+claimed, trip_id
player_photos	玩家照片卡、质量、首次拍照和相册进度。	photo_id	players, landmarks, player_trips	player_id+landmark_id, trip_id
weather_profiles	天气权重、距离 bucket 和倍率。	weather_profile_id	config_versions	config_version_id
daily_login_state	每日登录领取状态。	player_id + period_key	players	player_id+period_key
quest_definitions	每日/每周任务模板配置。	quest_id	config_versions	quest_type+period_scope+live_status
player_quest_progress	玩家任务进度和领取状态。	progress_id	players, quest_definitions	player_id+period_key
gacha_banners	Gacha banner 配置、价格和保底规则。	banner_id	config_versions	live_status+start_at+end_at
gacha_pool_items	Gacha 池内物品与权重。	pool_item_id	gacha_banners	banner_id+rarity+live_status
gacha_pity_state	玩家每个 banner 的保底状态。	player_id + banner_id	players, gacha_banners	player_id+banner_id
gacha_history	抽卡历史和审计。	gacha_history_id	players, gacha_banners	player_id+created_at, idempotency_key
config_versions	配置版本、发布、回滚和审计。	config_version_id	-	status+published_at
analytics_events	服务端埋点事件。	event_id	players nullable	event_name+event_time, player_id
suspicious_events	风险事件和反作弊日志。	suspicious_event_id	players	player_id+risk_type+created_at

表名	用途	主键	主要外键	索引建议
migration_log	配置迁移预览、执行和回滚日志。	migration_id	config_versions	status+executed_at

21.x players

项目	说明
用途	玩家账号、教程状态、风险状态和 last_seen_at。
主键	player_id
外键	-
索引建议	auth_provider+external_id, risk_status, last_seen_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE players (  
  player_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  auth_provider TEXT NOT NULL,  
  external_id TEXT NOT NULL,  
  display_name TEXT,  
  timezone TEXT NOT NULL DEFAULT 'UTC',  
  tutorial_state TEXT NOT NULL DEFAULT 'NOT_STARTED',  
  current_vehicle_id UUID,  
  risk_status TEXT NOT NULL DEFAULT 'NORMAL',  
  last_seen_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE(auth_provider, external_id)  
);  
CREATE INDEX idx_players_risk_status ON players(risk_status);  
CREATE INDEX idx_players_last_seen_at ON players(last_seen_at);
```

21.x wallet_balances

项目	说明
用途	货币余额快照。
主键	player_id + currency
外键	players
索引建议	player_id, currency
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE wallet_balances (  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  currency TEXT NOT NULL CHECK (currency IN  
( 'ROAD_COINS', 'TRAVEL_TOKENS', 'SOUVENIR_STAMPS', 'STAMP_FRAGMENTS', 'BLUEPRINTS' )),  
  balance BIGINT NOT NULL DEFAULT 0 CHECK (balance >= 0),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY(player_id, currency)  
);
```

21.x wallet_transactions

项目	说明
用途	不可变钱包流水，所有经济变化必须进入。
主键	transaction_id
外键	players
索引建议	player_id+created_at, source_type+source_id, idempotency_key
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE wallet_transactions (  
  transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  currency TEXT NOT NULL,  
  amount BIGINT NOT NULL,  
  balance_before BIGINT NOT NULL CHECK (balance_before >= 0),  
  balance_after BIGINT NOT NULL CHECK (balance_after >= 0),  
  reason TEXT NOT NULL,  
  source_type TEXT NOT NULL,  
  source_id TEXT,  
  idempotency_key TEXT NOT NULL,  
  metadata JSONB NOT NULL DEFAULT '{}':::jsonb,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE(player_id, idempotency_key)  
);  
CREATE INDEX idx_wallet_tx_player_created ON wallet_transactions(player_id, created_at DESC);  
CREATE INDEX idx_wallet_tx_source ON wallet_transactions(source_type, source_id);
```

21.x player_inventory

项目	说明
用途	通用库存、道具、蓝图分类库存。
主键	player_id + item_type + item_id
外键	players
索引建议	player_id+item_type
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE player_inventory (  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  item_type TEXT NOT NULL,  
  item_id TEXT NOT NULL,  
  quantity BIGINT NOT NULL DEFAULT 0 CHECK (quantity >= 0),  
  metadata JSONB NOT NULL DEFAULT '{}':::jsonb,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY(player_id, item_type, item_id)  
);  
CREATE INDEX idx_inventory_player_type ON player_inventory(player_id, item_type);
```

21.x vehicle_definitions

项目	说明
用途	车辆配置定义。
主键	vehicle_def_id
外键	config_versions
索引建议	config_version, rarity, live_status
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE vehicle_definitions (  
  vehicle_def_id UUID PRIMARY KEY,  
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  vehicle_key TEXT NOT NULL,  
  display_name TEXT NOT NULL,  
  rarity TEXT NOT NULL CHECK (rarity IN ('Common', 'Rare', 'Epic', 'Legendary')),  
  base_speed_kmph NUMERIC(8,2) NOT NULL,  
  fuel_capacity NUMERIC(8,2) NOT NULL,  
  fuel_consumption_per_km NUMERIC(8,4) NOT NULL,  
  durability NUMERIC(8,2) NOT NULL DEFAULT 100,  
  durability_loss_per_km NUMERIC(8,4) NOT NULL,  
  cleanliness_loss_per_km NUMERIC(8,4) NOT NULL,  
  offline_efficiency NUMERIC(5,3) NOT NULL CHECK (offline_efficiency BETWEEN 0 AND 1),  
  weather_resistance NUMERIC(5,3) NOT NULL CHECK (weather_resistance BETWEEN 0 AND 1),  
  default_skin_id TEXT,  
  metadata JSONB NOT NULL DEFAULT '{}':::jsonb,  
  UNIQUE(config_version_id, vehicle_key)  
);
```

21.x player_vehicles

项目	说明
用途	玩家拥有车辆与运行状态。
主键	player_vehicle_id
外键	players, vehicle_definitions
索引建议	player_id, selected unique
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE player_vehicles (  
  player_vehicle_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  vehicle_def_id UUID NOT NULL REFERENCES vehicle_definitions(vehicle_def_id),  
  current_fuel NUMERIC(8,2) NOT NULL,  
  current_durability NUMERIC(8,2) NOT NULL,  
  current_cleanliness NUMERIC(8,2) NOT NULL DEFAULT 100,  
  selected_skin_id TEXT,  
  upgrade_level INT NOT NULL DEFAULT 1,  
  total_distance_km NUMERIC(12,3) NOT NULL DEFAULT 0,  
  is_selected BOOLEAN NOT NULL DEFAULT FALSE,  
  locked_in_trip_id UUID,  
  version INT NOT NULL DEFAULT 1,  
  acquired_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE INDEX idx_player_vehicles_player ON player_vehicles(player_id);  
CREATE UNIQUE INDEX uq_selected_vehicle_per_player ON player_vehicles(player_id) WHERE is_selected = TRUE;
```

21.x route_definitions

项目	说明
用途	路线定义与解锁/奖励配置。
主键	route_id
外键	config_versions, weather_profiles
索引建议	route_type+is_active, config_version_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE route_definitions (  
  route_id UUID PRIMARY KEY,  
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  route_key TEXT NOT NULL,  
  name TEXT NOT NULL,  
  region TEXT NOT NULL,  
  start_node TEXT NOT NULL,  
  destination_node TEXT NOT NULL,  
  route_type TEXT NOT NULL CHECK (route_type IN ('Tutorial','Short','Medium','Long','Epic')),  
  total_distance_km NUMERIC(10,2) NOT NULL CHECK (total_distance_km > 0),  
  difficulty INT NOT NULL CHECK (difficulty BETWEEN 1 AND 5),  
  unlock_cost_stamps INT NOT NULL DEFAULT 0,  
  trip_prep_fee_coins INT NOT NULL DEFAULT 0,  
  reward_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,  
  weather_profile_id UUID REFERENCES weather_profiles(weather_profile_id),  
  day_night_profile JSONB NOT NULL DEFAULT '{}':::jsonb,  
  background_pack_id TEXT NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  is_deprecated BOOLEAN NOT NULL DEFAULT FALSE,  
  UNIQUE(config_version_id, route_key)  
);
```

21.x route_segments

项目	说明
用途	路线分段与地形/背景/倍率。
主键	segment_id
外键	route_definitions
索引建议	route_id+start_km+end_km
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE route_segments (  
  segment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  route_id UUID NOT NULL REFERENCES route_definitions(route_id) ON DELETE CASCADE,  
  segment_index INT NOT NULL,  
  start_km NUMERIC(10,2) NOT NULL,  
  end_km NUMERIC(10,2) NOT NULL,  
  terrain_type TEXT NOT NULL,  
  biome TEXT,  
  background_id TEXT,  
  speed_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,  
  fuel_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,  
  cleanliness_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,  
  durability_multiplier NUMERIC(6,3) NOT NULL DEFAULT 1.0,  
  UNIQUE(route_id, segment_index),  
  CHECK(end_km > start_km)
```

```
);
CREATE INDEX idx_segments_route_range ON route_segments(route_id, start_km, end_km);
```

21.x landmarks

项目	说明
用途	路线景点、强制停靠与照片卡配置。
主键	landmark_id
外键	route_definitions
索引建议	route_id+distance_km, album_group_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE landmarks (
  landmark_id UUID PRIMARY KEY,
  route_id UUID NOT NULL REFERENCES route_definitions(route_id),
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),
  name TEXT NOT NULL,
  distance_km NUMERIC(10,2) NOT NULL CHECK (distance_km >= 0),
  rarity TEXT NOT NULL CHECK (rarity IN ('Common','Rare','Epic','Legendary')),
  required_stop BOOLEAN NOT NULL DEFAULT TRUE,
  base_photo_coins INT NOT NULL DEFAULT 80,
  photo_card_key TEXT NOT NULL,
  album_group_id TEXT,
  metadata JSONB NOT NULL DEFAULT '{}'::jsonb
);
CREATE INDEX idx_landmarks_route_distance ON landmarks(route_id, distance_km);
```

21.x player_unlocked_routes

项目	说明
用途	玩家已永久解锁路线。
主键	player_id + route_id
外键	players, route_definitions
索引建议	player_id, route_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE player_unlocked_routes (
  player_id UUID NOT NULL REFERENCES players(player_id),
  route_id UUID NOT NULL REFERENCES route_definitions(route_id),
  unlocked_by TEXT NOT NULL,
  cost_stamps INT NOT NULL DEFAULT 0,
  unlocked_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY(player_id, route_id)
);
```

21.x player_trips

项目	说明
用途	当前/历史旅程主状态。

项目	说明
主键	trip_id
外键	players, route_definitions, player_vehicles
索引建议	player_id+status, route_id, last_simulated_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE player_trips (  
  trip_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  route_id UUID NOT NULL REFERENCES route_definitions(route_id),  
  route_config_version UUID NOT NULL REFERENCES config_versions(config_version_id),  
  player_vehicle_id UUID NOT NULL REFERENCES player_vehicles(player_vehicle_id),  
  status TEXT NOT NULL DEFAULT 'ACTIVE' CHECK (status IN  
( 'ACTIVE', 'PAUSED', 'FORCED_STOP', 'COMPLETED', 'ABANDONED' )),  
  current_distance_km NUMERIC(12,3) NOT NULL DEFAULT 0,  
  elapsed_real_seconds BIGINT NOT NULL DEFAULT 0,  
  online_token_meter_km NUMERIC(12,3) NOT NULL DEFAULT 0,  
  offline_token_meter_km NUMERIC(12,3) NOT NULL DEFAULT 0,  
  last_simulated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  completed_at TIMESTAMPTZ,  
  forced_stop_reason TEXT,  
  metadata JSONB NOT NULL DEFAULT '{}'::jsonb  
);  
CREATE INDEX idx_trips_player_status ON player_trips(player_id, status);
```

21.x offline_reports

项目	说明
用途	离线报告 pending 奖励与历史审计。
主键	report_id
外键	players, player_trips
索引建议	player_id+claimed, trip_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE offline_reports (  
  report_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  trip_id UUID NOT NULL REFERENCES player_trips(trip_id),  
  generated_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  offline_seconds BIGINT NOT NULL,  
  distance_travelled_km NUMERIC(12,3) NOT NULL,  
  road_coins_pending INT NOT NULL DEFAULT 0,  
  travel_tokens_pending INT NOT NULL DEFAULT 0,  
  fuel_used NUMERIC(10,3) NOT NULL DEFAULT 0,  
  cleanliness_loss NUMERIC(10,3) NOT NULL DEFAULT 0,  
  durability_loss NUMERIC(10,3) NOT NULL DEFAULT 0,  
  weather_summary JSONB NOT NULL DEFAULT '{}'::jsonb,  
  landmark_reached JSONB,  
  forced_stop_reason TEXT,  
  claimed BOOLEAN NOT NULL DEFAULT FALSE,  
  claimed_at TIMESTAMPTZ,  
  claim_idempotency_key TEXT  
);  
CREATE INDEX idx_offline_reports_player_claimed ON offline_reports(player_id, claimed, generated_at DESC);  
CREATE UNIQUE INDEX uq_one_pending_report_per_trip ON offline_reports(player_id, trip_id) WHERE claimed = FALSE;
```

21.x player_photos

项目	说明
用途	玩家照片卡、质量、首次拍照和相册进度。
主键	photo_id
外键	players, landmarks, player_trips
索引建议	player_id+landmark_id, trip_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE player_photos (  
  photo_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  trip_id UUID NOT NULL REFERENCES player_trips(trip_id),  
  landmark_id UUID NOT NULL REFERENCES landmarks(landmark_id),  
  photo_card_key TEXT NOT NULL,  
  quality_score INT NOT NULL CHECK (quality_score BETWEEN 0 AND 100),  
  weather TEXT NOT NULL,  
  day_phase TEXT NOT NULL,  
  cleanliness_at_shot NUMERIC(5,2) NOT NULL,  
  is_first_photo BOOLEAN NOT NULL,  
  reward_tx_id UUID REFERENCES wallet_transactions(transaction_id),  
  taken_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  metadata JSONB NOT NULL DEFAULT '{}':::jsonb  
);  
CREATE UNIQUE INDEX uq_first_photo_per_landmark ON player_photos(player_id, landmark_id) WHERE is_first_photo = TRUE;
```

21.x weather_profiles

项目	说明
用途	天气权重、距离 bucket 和倍率。
主键	weather_profile_id
外键	config_versions
索引建议	config_version_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE weather_profiles (  
  weather_profile_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  seed_salt TEXT NOT NULL,  
  refresh_distance_km NUMERIC(8,2) NOT NULL DEFAULT 25 CHECK (refresh_distance_km > 0),  
  weights JSONB NOT NULL,  
  effects JSONB NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

21.x daily_login_state

项目	说明
用途	每日登录领取状态。

项目	说明
主键	player_id + period_key
外键	players
索引建议	player_id+period_key
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE daily_login_state (  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  period_key TEXT NOT NULL,  
  cycle_day INT NOT NULL CHECK (cycle_day BETWEEN 1 AND 7),  
  claimed BOOLEAN NOT NULL DEFAULT FALSE,  
  reward_snapshot JSONB NOT NULL DEFAULT '{}':::jsonb,  
  claimed_at TIMESTAMPTZ,  
  idempotency_key TEXT,  
  PRIMARY KEY(player_id, period_key)  
);  
CREATE UNIQUE INDEX uq_daily_login_idem ON daily_login_state(player_id, idempotency_key) WHERE idempotency_key IS NOT NULL;
```

21.x quest_definitions

项目	说明
用途	每日/每周任务模板配置。
主键	quest_id
外键	config_versions
索引建议	quest_type+period_scope+live_status
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE quest_definitions (  
  quest_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  quest_key TEXT NOT NULL,  
  quest_type TEXT NOT NULL CHECK (quest_type IN ('Daily','Weekly')),  
  period_scope TEXT NOT NULL CHECK (period_scope IN ('DAY','WEEK')),  
  event_name TEXT NOT NULL,  
  target_value NUMERIC(12,3) NOT NULL,  
  reward JSONB NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  UNIQUE(config_version_id, quest_key)  
);
```

21.x player_quest_progress

项目	说明
用途	玩家任务进度和领取状态。
主键	progress_id
外键	players, quest_definitions
索引建议	player_id+period_key

项目	说明
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。
<pre>CREATE TABLE player_quest_progress (progress_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), player_id UUID NOT NULL REFERENCES players(player_id), quest_id UUID NOT NULL REFERENCES quest_definitions(quest_id), period_key TEXT NOT NULL, current_value NUMERIC(12,3) NOT NULL DEFAULT 0, completed BOOLEAN NOT NULL DEFAULT FALSE, completed_at TIMESTAMPTZ, claimed BOOLEAN NOT NULL DEFAULT FALSE, claimed_at TIMESTAMPTZ, idempotency_key TEXT, UNIQUE(player_id, quest_id, period_key));</pre>	

21.x gacha_banners

项目	说明
用途	Gacha banner 配置、价格和保底规则。
主键	banner_id
外键	config_versions
索引建议	live_status+start_at+end_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。
<pre>CREATE TABLE gacha_banners (banner_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id), banner_key TEXT NOT NULL, name TEXT NOT NULL, single_cost_tokens INT NOT NULL DEFAULT 20, ten_cost_tokens INT NOT NULL DEFAULT 180, rarity_rates JSONB NOT NULL, pity_rules JSONB NOT NULL, start_at TIMESTAMPTZ, end_at TIMESTAMPTZ, is_active BOOLEAN NOT NULL DEFAULT TRUE, UNIQUE(config_version_id, banner_key));</pre>	

21.x gacha_pool_items

项目	说明
用途	Gacha 池内物品与权重。
主键	pool_item_id
外键	gacha_banners
索引建议	banner_id+rarity+live_status
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。

项目	说明
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。
<pre>CREATE TABLE gacha_pool_items (pool_item_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), banner_id UUID NOT NULL REFERENCES gacha_banners(banner_id), item_type TEXT NOT NULL, item_id TEXT NOT NULL, rarity TEXT NOT NULL CHECK (rarity IN ('Common','Rare','Epic','Legendary')), weight INT NOT NULL CHECK (weight > 0), duplicate_blueprints INT NOT NULL DEFAULT 0, is_active BOOLEAN NOT NULL DEFAULT TRUE);</pre>	

21.x gacha_pity_state

项目	说明
用途	玩家每个 banner 的保底状态。
主键	player_id + banner_id
外键	players, gacha_banners
索引建议	player_id+banner_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。
<pre>CREATE TABLE gacha_pity_state (player_id UUID NOT NULL REFERENCES players(player_id), banner_id UUID NOT NULL REFERENCES gacha_banners(banner_id), pulls_since_rare INT NOT NULL DEFAULT 0, pulls_since_epic INT NOT NULL DEFAULT 0, pulls_sinceLegendary INT NOT NULL DEFAULT 0, total_pulls INT NOT NULL DEFAULT 0, updated_at TIMESTAMPTZ NOT NULL DEFAULT now(), PRIMARY KEY(player_id, banner_id));</pre>	

21.x gacha_history

项目	说明
用途	抽卡历史和审计。
主键	gacha_history_id
外键	players, gacha_banners
索引建议	player_id+created_at, idempotency_key
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。
<pre>CREATE TABLE gacha_history (gacha_history_id UUID PRIMARY KEY DEFAULT gen_random_uuid(), player_id UUID NOT NULL REFERENCES players(player_id), banner_id UUID NOT NULL REFERENCES gacha_banners(banner_id), pull_group_id UUID NOT NULL, pull_index INT NOT NULL, item_type TEXT NOT NULL, item_id TEXT NOT NULL, rarity TEXT NOT NULL,</pre>	

```

is_pity_triggered BOOLEAN NOT NULL DEFAULT FALSE,
duplicate_converted_to_blueprints INT NOT NULL DEFAULT 0,
idempotency_key TEXT NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE(player_id, idempotency_key, pull_index)
);

```

21.x config_versions

项目	说明
用途	配置版本、发布、回滚和审计。
主键	config_version_id
外键	-
索引建议	status+published_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```

CREATE TABLE config_versions (
  config_version_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  version_number INT NOT NULL UNIQUE,
  status TEXT NOT NULL CHECK (status IN ('DRAFT', 'VALIDATED', 'LIVE', 'DEPRECATED', 'ROLLED_BACK')),
  checksum TEXT NOT NULL,
  notes TEXT,
  created_by TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_by TEXT,
  published_at TIMESTAMPTZ,
  deprecated_at TIMESTAMPTZ,
  rollback_from UUID REFERENCES config_versions(config_version_id),
  validation_report JSONB NOT NULL DEFAULT '{}'::jsonb
);

```

21.x analytics_events

项目	说明
用途	服务端埋点事件。
主键	event_id
外键	players nullable
索引建议	event_name+event_time, player_id
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```

CREATE TABLE analytics_events (
  event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  player_id UUID REFERENCES players(player_id),
  event_name TEXT NOT NULL,
  event_time TIMESTAMPTZ NOT NULL DEFAULT now(),
  session_id TEXT,
  trip_id UUID,
  route_id UUID,
  payload JSONB NOT NULL DEFAULT '{}'::jsonb,
  server_version TEXT,
  config_version_id UUID
);
CREATE INDEX idx_analytics_event_time ON analytics_events(event_name, event_time DESC);

```

21.x suspicious_events

项目	说明
用途	风险事件和反作弊日志。
主键	suspicious_event_id
外键	players
索引建议	player_id+risk_type+created_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE suspicious_events (  
  suspicious_event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  player_id UUID NOT NULL REFERENCES players(player_id),  
  risk_type TEXT NOT NULL,  
  severity INT NOT NULL DEFAULT 1 CHECK (severity BETWEEN 1 AND 5),  
  source_endpoint TEXT,  
  trip_id UUID,  
  request_payload JSONB NOT NULL DEFAULT '{}':::jsonb,  
  server_snapshot JSONB NOT NULL DEFAULT '{}':::jsonb,  
  action_taken TEXT NOT NULL DEFAULT 'LOG_ONLY',  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

21.x migration_log

项目	说明
用途	配置迁移预览、执行和回滚日志。
主键	migration_id
外键	config_versions
索引建议	status+executed_at
关键字段解释	业务状态字段必须由后端事务更新；配置字段必须受 config_versions 保护。
关系说明	通过外键与玩家、配置、旅程或钱包流水建立可审计关系。

```
CREATE TABLE migration_log (  
  migration_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  from_config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  to_config_version_id UUID NOT NULL REFERENCES config_versions(config_version_id),  
  migration_type TEXT NOT NULL,  
  affected_players INT NOT NULL DEFAULT 0,  
  preview_report JSONB NOT NULL DEFAULT '{}':::jsonb,  
  executed_by TEXT,  
  executed_at TIMESTAMPTZ,  
  status TEXT NOT NULL DEFAULT 'PREVIEW',  
  error_summary JSONB NOT NULL DEFAULT '{}':::jsonb  
);
```

21.2 数据库事务边界与一致性要求

- 钱包扣费和发奖必须与业务状态更新在同一个数据库事务内完成。
- player_trips 在线 tick、离线模拟、路线完成必须使用 SELECT ... FOR UPDATE 锁定当前 trip。
- player_vehicles 维护状态更新必须和对应扣费流水同事务。
- gacha_pity_state 必须在抽卡事务中锁定；抽卡历史与钱包扣费同事务。
- 配置表发布后不可原地修改；新版本通过 config_versions 发布，旧版本标记为 DEPRECATED。

- 所有客户端可重试接口必须接收或生成 idempotency_key。

22. REST API 设计

V1 API 采用 REST 风格。所有写接口必须有后端校验和幂等策略。客户端只能提交玩家意图，不能提交距离、奖励、抽卡结果、离线时长或钱包变化。

```
{
  "success": true,
  "data": {},
  "error": null,
  "server_time": "2026-05-15T00:00:00Z",
  "request_id": "req_..."
}
```

22.1 API 总览

模块	方法	路径	用途	关联表
Player	GET	/player/profile	读取玩家基础资料、教程状态、风险状态、当前选中车辆摘要。	players, player_vehicles
Player	GET	/player/state	读取客户端首页聚合状态：钱包、车辆、当前 trip、pending offline_report、任务摘要。	players, wallet_balances, player_trips, offline_reports
Player	POST	/player/select-vehicle	选择默认车辆。	player_vehicles, players
Routes	GET	/routes/available	读取可见路线、解锁状态、解锁成本、预计时长。	route_definitions, player_unlocked_routes
Routes	GET	/routes/:route_id	读取路线详情、路段摘要、景点列表、天气/昼夜 profile 摘要。	route_definitions, route_segments, landmarks
Routes	POST	/routes/unlock	使用 Souvenir Stamps 永久解锁路线。	player_unlocked_routes, wallet_transactions
Routes	POST	/routes/start	开始旅程并扣 Trip Prep Fee。	player_trips, player_vehicles, wallet_transactions
Routes	POST	/routes/abandon	放弃当前旅程，不退还 Trip Prep Fee。	player_trips, analytics_events
Trip	GET	/trip/current	读取当前旅程完整状态。	player_trips, route_definitions, landmarks
Trip	POST	/trip/drive-tick	在线驾驶 tick，后端计算距离、消耗和奖励。	player_trips, player_vehicles, wallet_transactions
Trip	POST	/trip/claim-offline-report	领取离线报告 pending 奖励。	offline_reports, wallet_transactions

模块	方法	路径	用途	关联表
Trip	POST	/trip/complete-landmark	完成强制停靠景点并拍照。	player_trips, landmarks, player_photos
Trip	POST	/trip/complete-route	路线完成结算。	player_trips, wallet_transactions, analytics_events
Economy	GET	/wallet	读取钱包余额。	wallet_balances
Economy	GET	/wallet/transactions	分页读取钱包流水。	wallet_transactions
Vehicle Maintenance	POST	/vehicle/refuel	给车辆加油并扣 Road Coins。	player_vehicles, wallet_transactions
Vehicle Maintenance	POST	/vehicle/clean	洗车并恢复清洁度。	player_vehicles, wallet_transactions
Vehicle Maintenance	POST	/vehicle/repair	修理车辆并恢复耐久。	player_vehicles, wallet_transactions
Gacha	GET	/gacha/banners	读取当前可用 banner 与价格、概率展示。	gacha_banners, gacha_pool_items
Gacha	POST	/gacha/pull	后端抽卡、扣 Travel Tokens、更新保底、写历史。	gacha_pity_state, gacha_history, wallet_transactions
Gacha	GET	/gacha/history	分页读取抽卡历史。	gacha_history
Inventory	GET	/inventory	读取库存。	player_inventory
Inventory	POST	/inventory/use-item	使用道具，例如蓝图兑换或礼包开启。	player_inventory, wallet_transactions
Daily / Quest	GET	/daily-login	读取今日登录奖励状态。	daily_login_state
Daily / Quest	POST	/daily-login/claim	领取每日登录奖励。	daily_login_state, wallet_transactions
Daily / Quest	GET	/quests/daily	读取每日任务进度。	quest_definitions, player_quest_progress
Daily / Quest	POST	/quests/claim	领取任务奖励。	player_quest_progress, wallet_transactions
Admin	GET	/admin/configs	后台读取配置版本列表和状态。	config_versions
Admin	POST	/admin/configs/draft	创建配置草稿。	config_versions
Admin	POST	/admin/configs/validate	校验草稿配置完整性和经济模拟。	config_versions, route_definitions
Admin	POST	/admin/configs/publish	发布配置为 live。	config_versions
Admin	POST	/admin/configs/rollback	回滚到旧 live/deprecated 配置。	config_versions

22.x GET /player/profile

项目	说明
用途	读取玩家基础资料、教程状态、风险状态、当前选中车辆摘要。
请求参数	无
返回示例	profile JSON
后端校验	鉴权 token 有效；玩家存在。
幂等要求	GET 无幂等键。
错误码	401 UNAUTHORIZED; 404 PLAYER_NOT_FOUND
关联数据表	players, player_vehicles

22.x GET /player/state

项目	说明
用途	读取客户端首屏聚合状态：钱包、车辆、当前 trip、pending offline_report、任务摘要。
请求参数	无
返回示例	state JSON
后端校验	若有 active trip，先查询 pending report；已有 pending report 不重复生成。
幂等要求	GET 无幂等键。
错误码	401; 404
关联数据表	players, wallet_balances, player_trips, offline_reports

22.x POST /player/select-vehicle

项目	说明
用途	选择默认车辆。
请求参数	player_vehicle_id
返回示例	selected_vehicle_id
后端校验	车辆属于玩家；旅程中不可换车。
幂等要求	同一车辆重复选择返回成功。
错误码	400 INVALID_VEHICLE; 409 VEHICLE_LOCKED_IN_TRIP
关联数据表	player_vehicles, players

22.x GET /routes/available

项目	说明
用途	读取可见路线、解锁状态、解锁成本、预计时长。
请求参数	route_type, include_locked
返回示例	routes[]
后端校验	只返回 live config；教程未完成时只返回 Tutorial。
幂等要求	GET 无。
错误码	401; 400 INVALID_FILTER
关联数据表	route_definitions, player_unlocked_routes

22.x GET /routes/:route_id

项目	说明
用途	读取路线详情、路段摘要、景点列表、天气/昼夜 profile 摘要。
请求参数	route_id
返回示例	route detail
后端校验	route 存在且可见；隐藏未发布配置。
幂等要求	GET 无。
错误码	404 ROUTE_NOT_FOUND
关联数据表	route_definitions, route_segments, landmarks

22.x POST /routes/unlock

项目	说明
用途	使用 Souvenir Stamps 永久解锁路线。
请求参数	route_id, idempotency_key
返回示例	unlock result
后端校验	非 Tutorial；未解锁；余额足够；成本按配置。
幂等要求	必需；重复 key 返回首次结果。
错误码	400 ALREADY_UNLOCKED; 402 INSUFFICIENT_STAMPS
关联数据表	player_unlocked_routes, wallet_transactions

22.x POST /routes/start

项目	说明
用途	开始旅程并扣 Trip Prep Fee。
请求参数	route_id, player_vehicle_id, idempotency_key
返回示例	trip state
后端校验	路线已解锁；无 active trip；车辆可用；油量 > 0；Road Coins 足够。
幂等要求	必需；重复 key 返回同一 trip。
错误码	402 INSUFFICIENT_COINS; 409 ACTIVE_TRIP_EXISTS; 409 LOW_FUEL
关联数据表	player_trips, player_vehicles, wallet_transactions

22.x POST /routes/abandon

项目	说明
用途	放弃当前旅程，不退还 Trip Prep Fee。
请求参数	trip_id, reason, idempotency_key
返回示例	status=ABANDONED
后端校验	trip 属于玩家且未完成。
幂等要求	必需；重复 key 返回已放弃状态。
错误码	404 TRIP_NOT_FOUND; 409 TRIP_ALREADY_COMPLETED
关联数据表	player_trips, analytics_events

22.x GET /trip/current

项目	说明
用途	读取当前旅程完整状态。
请求参数	无
返回示例	current_trip
后端校验	鉴权；若无 active trip 返回 null；返回 pending report。
幂等要求	GET 无。
错误码	401
关联数据表	player_trips, route_definitions, landmarks

22.x POST /trip/drive-tick

项目	说明
用途	在线驾驶 tick，后端计算距离、消耗和奖励。
请求参数	trip_id, mode, client_tick_seq, idempotency_key
返回示例	simulation result
后端校验	trip active; tick 间隔合法; mode 已解锁; 客户端不传距离或奖励。
幂等要求	必需; 按 player+key 唯一。
错误码	400 INVALID_MODE; 409 FORCED_STOP; 429 TICK_RATE_LIMITED
关联数据表	player_trips, player_vehicles, wallet_transactions

22.x POST /trip/claim-offline-report

项目	说明
用途	领取离线报告 pending 奖励。
请求参数	report_id, idempotency_key
返回示例	claimed rewards
后端校验	report 属于玩家; claimed=false; SELECT FOR UPDATE。
幂等要求	必需; 重复 key 返回首次领取结果。
错误码	404 REPORT_NOT_FOUND; 409 REPORT_ALREADY_CLAIMED
关联数据表	offline_reports, wallet_transactions

22.x POST /trip/complete-landmark

项目	说明
用途	完成强制停靠景点并拍照。
请求参数	trip_id, landmark_id, action, idempotency_key
返回示例	photo_card, rewards
后端校验	trip 已到达景点距离; landmark 属于路线; 不能提前完成。
幂等要求	trip+landmark 幂等。
错误码	400 LANDMARK_NOT_REACHED; 404
关联数据表	player_trips, landmarks, player_photos

22.x POST /trip/complete-route

项目	说明
用途	路线完成结算。
请求参数	trip_id, idempotency_key
返回示例	completion rewards
后端校验	距离到达终点；未结算过；强制景点已处理。
幂等要求	必需；重复 key 返回首次结果。
错误码	409 ROUTE_NOT_FINISHED; 409 ALREADY_COMPLETED
关联数据表	player_trips, wallet_transactions, analytics_events

22.x GET /wallet

项目	说明
用途	读取钱包余额。
请求参数	无
返回示例	balances
后端校验	鉴权。
幂等要求	GET 无。
错误码	401
关联数据表	wallet_balances

22.x GET /wallet/transactions

项目	说明
用途	分页读取钱包流水。
请求参数	currency, cursor, limit
返回示例	items, next_cursor
后端校验	limit <= 100；仅玩家本人流水。
幂等要求	GET 无。
错误码	400 INVALID_PAGING
关联数据表	wallet_transactions

22.x POST /vehicle/refuel

项目	说明
用途	给车辆加油并扣 Road Coins。
请求参数	player_vehicle_id, fuel_amount, idempotency_key
返回示例	fuel, cost
后端校验	车辆属于玩家；不超过油箱；余额足够。
幂等要求	必需。
错误码	402 INSUFFICIENT_COINS; 400 FUEL_OVER_CAPACITY
关联数据表	player_vehicles, wallet_transactions

22.x POST /vehicle/clean

项目	说明
用途	洗车并恢复清洁度。
请求参数	player_vehicle_id, clean_points, idempotency_key
返回示例	cleanliness, cost
后端校验	车辆属于玩家；clean_points 合法；余额足够。
幂等要求	必需。
错误码	402; 400 ALREADY_CLEAN
关联数据表	player_vehicles, wallet_transactions

22.x POST /vehicle/repair

项目	说明
用途	修理车辆并恢复耐久。
请求参数	player_vehicle_id, repair_points, idempotency_key
返回示例	durability, cost
后端校验	车辆属于玩家；repair_points 合法；余额足够。
幂等要求	必需。
错误码	402; 400 ALREADY_FULL_DURABILITY
关联数据表	player_vehicles, wallet_transactions

22.x GET /gacha/banners

项目	说明
用途	读取当前可用 banner 与价格、概率展示。
请求参数	无
返回示例	banners[]
后端校验	只返回 live 且在时间窗内 banner。
幂等要求	GET 无。
错误码	401
关联数据表	gacha_banners, gacha_pool_items

22.x POST /gacha/pull

项目	说明
用途	后端抽卡、扣 Travel Tokens、更新保底、写历史。
请求参数	banner_id, pull_count, idempotency_key
返回示例	results, pity_after
后端校验	pull_count 只能 1 或 10；余额足够；客户端不能传随机数或结果。
幂等要求	必需。
错误码	402 INSUFFICIENT_TOKENS; 400 INVALID_PULL_COUNT
关联数据表	gacha_pity_state, gacha_history, wallet_transactions

22.x GET /gacha/history

项目	说明
用途	分页读取抽卡历史。
请求参数	banner_id, cursor
返回示例	items
后端校验	仅玩家本人。
幂等要求	GET 无。
错误码	400 INVALID_PAGING
关联数据表	gacha_history

22.x GET /inventory

项目	说明
用途	读取库存。
请求参数	item_type
返回示例	items[]
后端校验	鉴权。
幂等要求	GET 无。
错误码	401
关联数据表	player_inventory

22.x POST /inventory/use-item

项目	说明
用途	使用道具，例如蓝图兑换或礼包开启。
请求参数	item_type, item_id, quantity, idempotency_key
返回示例	used result
后端校验	库存足够；物品可使用；结果由配置决定。
幂等要求	必需。
错误码	402 INSUFFICIENT_ITEM; 400 ITEM_NOT_USABLE
关联数据表	player_inventory, wallet_transactions

22.x GET /daily-login

项目	说明
用途	读取今日登录奖励状态。
请求参数	无
返回示例	period_key, claimed, cycle_day
后端校验	按运营时区生成 period_key。
幂等要求	GET 无。
错误码	401
关联数据表	daily_login_state

22.x POST /daily-login/claim

项目	说明
用途	领取每日登录奖励。
请求参数	idempotency_key
返回示例	claimed reward
后端校验	今日未领取；锁定 daily_login_state；Day 7 Stamp 每周最多 1 个。
幂等要求	必需。
错误码	409 ALREADY_CLAIMED
关联数据表	daily_login_state, wallet_transactions

22.x GET /quests/daily

项目	说明
用途	读取每日任务进度。
请求参数	无
返回示例	quests[]
后端校验	确保 period progress 初始化。
幂等要求	GET 无。
错误码	401
关联数据表	quest_definitions, player_quest_progress

22.x POST /quests/claim

项目	说明
用途	领取任务奖励。
请求参数	quest_id, period_key, idempotency_key
返回示例	claimed reward
后端校验	任务完成且未领取；progress 属于玩家。
幂等要求	必需。
错误码	409 QUEST_NOT_COMPLETED; 409 ALREADY_CLAIMED
关联数据表	player_quest_progress, wallet_transactions

22.x GET /admin/configs

项目	说明
用途	后台读取配置版本列表和状态。
请求参数	status
返回示例	configs[]
后端校验	管理员权限；支持分页。
幂等要求	GET 无。
错误码	403 FORBIDDEN
关联数据表	config_versions

22.x POST /admin/configs/draft

项目	说明
用途	创建配置草稿。
请求参数	base_config_version, notes
返回示例	draft version
后端校验	管理员权限；base config 存在。
幂等要求	后台 request_id 防重复。
错误码	403; 404 BASE_CONFIG_NOT_FOUND
关联数据表	config_versions

22.x POST /admin/configs/validate

项目	说明
用途	校验草稿配置完整性和经济模拟。
请求参数	config_version
返回示例	validation_report
后端校验	草稿存在；校验 routes/segments/economy/gacha。
幂等要求	同版本重校验覆盖 validation_report。
错误码	400 CONFIG_VALIDATION_FAILED
关联数据表	config_versions, route_definitions

22.x POST /admin/configs/publish

项目	说明
用途	发布配置为 live。
请求参数	config_version, publish_note
返回示例	LIVE version
后端校验	必须 VALIDATED；同一时间只允许一个 live。
幂等要求	需要发布锁。
错误码	409 CONFIG_NOT_VALIDATED; 409 PUBLISH_LOCKED
关联数据表	config_versions

22.x POST /admin/configs/rollback

项目	说明
用途	回滚到旧 live/deprecated 配置。
请求参数	target_config_version, reason
返回示例	new live version
后端校验	目标版本完整；生成新版本而不是直接复活旧行。
幂等要求	后台 request_id。
错误码	404 TARGET_NOT_FOUND; 409 ROLLBACK_VALIDATION_FAILED
关联数据表	config_versions

22.2 后端统一校验规则

鉴权、所有权、配置状态、钱包余额、幂等键、速率限制和风险记录必须统一处理。写接口需要 idempotency_key；drive-tick、gacha-pull、claim 类接口使用 Redis 短窗口限流。疑似作弊请求写 suspicious_events。

22.3 推荐错误码

错误码	HTTP	说明
UNAUTHORIZED	401	未登录或 token 无效
FORBIDDEN	403	权限不足
PLAYER_NOT_FOUND	404	玩家不存在
ROUTE_NOT_FOUND	404	路线不存在或不可见
ACTIVE_TRIP_EXISTS	409	玩家已有进行中的旅程

错误码	HTTP	说明
TRIP_NOT_FOUND	404	旅程不存在或不属于玩家
TRIP_ALREADY_COMPLETED	409	旅程已完成
ROUTE_NOT_FINISHED	409	未到路线终点
INSUFFICIENT_COINS	402	Road Coins 不足
INSUFFICIENT_TOKENS	402	Travel Tokens 不足
INSUFFICIENT_STAMPS	402	Souvenir Stamps 不足
INSUFFICIENT_ITEM	402	道具不足
LOW_FUEL	409	油量不足
FORCED_STOP	409	当前处于强制停靠状态
TICK_RATE_LIMITED	429	drive tick 过于频繁
INVALID_MODE	400	驾驶模式无效或未解锁
CONFIG_VALIDATION_FAILED	400	后台配置校验失败
IDEMPOTENCY_CONFLICT	409	同一幂等键请求体不一致

23. Trip Simulation Engine

Trip Simulation Engine 是 V1 最重要的技术地基。它不是前端动画系统，而是后端权威的旅程状态推进器。所有行驶距离、油耗、耐久、清洁度、景点停靠、路线完成、在线/离线奖励，都应由该引擎计算并写入数据库。

23.1 模块职责、输入、输出和依赖

职责	说明	落地要求
在线 tick 模拟	根据服务器时间、车辆属性、路线段、天气、驾驶模式计算距离和消耗。	客户端不传距离；服务器限制 tick 最大时长。
离线推进模拟	玩家回归时按服务器时间差计算离线距离和 pending 奖励。	max_offline_hours=8；奖励低于在线。
forced stop	低油、强制景点、路线终点、异常状态时停止推进。	写入 trip.status / forced_stop_reason。
奖励发放	在线奖励即时入账；离线奖励先进入 offline_reports pending。	所有入账写 wallet_transactions。
并发控制	同一 trip 同一时刻只能被一个请求推进。	SELECT FOR UPDATE + idempotency。

类别	字段	来源	说明
输入	player_id	Auth	当前玩家。

类别	字段	来源	说明
输入	trip_id	Client	玩家意图对应旅程。
输入	mode	Client	Hold to Drive / Auto Driving / Hold to Boost。
输入	idempotency_key	Client	写接口重试去重。
依赖	player_trips	DB	当前距离、版本、状态、last_simulated_at。
依赖	player_vehicles	DB	油量、耐久、清洁度、属性。
依赖	route_definitions / route_segments	DB/Config	路线总长、路段倍率。
依赖	weather_profiles	DB/Config	天气权重和倍率。
输出	distance_gain_km	Response/DB	本次推进距离。
输出	vehicle_state_delta	Response/DB	油量、清洁、耐久变化。
输出	rewards	Response/DB	本次发放或 pending 奖励。
输出	forced_stop_reason	Response/DB	停止原因。

23.2 核心公式

```
distance_gain = vehicle_speed_kmph * duration_seconds / 3600 * mode_multiplier * condition_multiplier *
segment_speed_multiplier
condition_multiplier = durability_speed_multiplier * weather_speed_multiplier * fuel_state_multiplier
fuel_used = distance_km * vehicle.fuel_consumption_per_km * weather_fuel_multiplier * terrain_fuel_multiplier *
durability_fuel_penalty * mode_fuel_multiplier
cleanliness_loss = distance_km * vehicle.cleanliness_loss_per_km * weather_cleanliness_multiplier *
terrain_cleanliness_multiplier
durability_loss = distance_km * vehicle.durability_loss_per_km * terrain_durability_multiplier *
adjusted_weather_durability_multiplier
adjusted_weather_durability_multiplier = 1 + (weather_durability_multiplier - 1) * (1 -
vehicle.weather_resistance)
```

23.3 forced stop logic

停止原因	触发条件	后端动作	UI 建议
LOW_FUEL	油量不足以继续推进。	trip.status=FORCED_STOP。	提示加油或使用补给。
LANDMARK_REQUIRED	到达 required landmark。	停止在景点距离。	展示景点停靠和拍照按钮。
ROUTE_END	达到路线总长。	允许 complete-route。	展示路线完成结算。
VEHICLE_BROKEN	耐久低于硬阈值。	FORCED_STOP。	提示修理。
CONFIG_MISSING	路线段或天气配置缺失。	暂停并记录风险。	提示稍后重试。
RISK_LIMITED	玩家 risk_status=LIMITED。	限制奖励或拒绝 tick。	提示账号状态异常。

23.4 wallet reward logic

在线奖励可以即时写入钱包；离线奖励必须先 pending 到 report。所有发奖 reason 需要明确，例如 ONLINE_DRIVE_REWARD、OFFLINE_REPORT_CLAIM、ROUTE_COMPLETE_REWARD、PHOTO_FIRST_REWARD。

```
online_road_coins = floor(online_distance_km * 10 * route_reward_multiplier * active_event_multiplier)
offline_road_coins = floor(offline_distance_km * 4 * route_reward_multiplier)
```

23.5 transaction boundary、concurrency control、idempotency、risk logging

每次在线 tick、离线报告生成、离线报告领取、路线完成结算都是独立事务。关键锁：player_trips FOR UPDATE、player_vehicles FOR UPDATE、wallet_balances FOR UPDATE、gacha_pity_state FOR UPDATE。重复请求通过 idempotency_key 返回首次结果。非法 mode、异常 tick、速度超过上限、重复领取等写 suspicious_events。

23.6 工程伪代码

```
function calculate_distance_gain(vehicle, segment, weather, mode, duration_seconds):
    mode_multiplier = get_mode_multiplier(mode)
    condition_multiplier = get_durability_speed_multiplier(vehicle.current_durability) * weather.speed *
    fuel_state_multiplier(vehicle.current_fuel)
    return max(vehicle.base_speed_kmph * duration_seconds / 3600 * mode_multiplier * condition_multiplier *
    segment.speed_multiplier, 0)

function calculate_fuel_used(distance_km, vehicle, segment, weather, mode):
    return max(distance_km * vehicle.fuel_consumption_per_km * weather.fuel * segment.fuel_multiplier *
    get_durability_fuel_penalty(vehicle.current_durability) * get_mode_fuel_multiplier(mode), 0)

function calculate_cleanliness_loss(distance_km, vehicle, segment, weather):
    return clamp(distance_km * vehicle.cleanliness_loss_per_km * weather.cleanliness *
    segment.cleanliness_multiplier, 0, vehicle.current_cleanliness)

function calculate_durability_loss(distance_km, vehicle, segment, weather):
    adjusted_weather = 1 + (weather.durability - 1) * (1 - vehicle.weather_resistance)
    return clamp(distance_km * vehicle.durability_loss_per_km * segment.durability_multiplier * adjusted_weather,
    0, vehicle.current_durability)

function check_for_forced_stop(trip, vehicle, route, landmarks, effective_fuel_per_km):
    if vehicle.current_fuel <= 0: return ForcedStop('LOW_FUEL', 0)
    route_remaining = route.total_distance_km - trip.current_distance_km
    distance_to_landmark = distance_to_next_required_landmark(landmarks, trip.current_distance_km)
    fuel_limited_distance = vehicle.current_fuel / effective_fuel_per_km
    distance_limit = min(route_remaining, distance_to_landmark, fuel_limited_distance)
    if distance_limit == fuel_limited_distance: return ForcedStop('LOW_FUEL', distance_limit)
    if distance_limit == distance_to_landmark: return ForcedStop('LANDMARK_REQUIRED', distance_limit)
    if distance_limit == route_remaining: return ForcedStop('ROUTE_END', distance_limit)
    return ForcedStop(null, distance_limit)

function grant_trip_rewards(tx, player_id, trip, distance_km, kind, idempotency_key):
    if kind == 'ONLINE':
        coins = floor(distance_km * 10 * trip.route.reward_multiplier)
        wallet.add(tx, player_id, 'ROAD_COINS', coins, 'ONLINE_DRIVE_REWARD', 'TRIP', trip.trip_id, idempotency_key +
        ':coins')
        accumulate online_token_meter, grant 1 token per 10 km
    if kind == 'OFFLINE':
        coins = floor(distance_km * 4 * trip.route.reward_multiplier)
        accumulate offline_token_meter, pending 1 token per 20 km
    return pending_rewards
```

simulate_online_tick() 和 simulate_offline_progress() 的完整流程见第 13 与第 14 章；实现时必须把上述函数作为可单元测试的纯函数优先落地。

24. 反作弊和风险控制

V1 的反作弊目标不是立刻构建复杂封号系统，而是保证核心经济和旅程推进不被客户端随意篡改。所有权威计算必须在后端完成；所有异常行为必须有日志；对轻微异常先限流、标记和客服可见，不建议自动封号。

不信任对象	风险	后端策略
客户端时间	伪造离线时长、加速 tick。	使用 server_now；离线从 player.last_seen_at / trip.last_simulated_at 计算。

不信任对象	风险	后端策略
客户端距离	直接上报超长距离刷奖励。	客户端不允许提交 distance；后端按公式计算。
客户端奖励	伪造金币、Token、Stamps。	所有变化由 Wallet Service 写流水。
客户端抽卡结果	伪造稀有车辆或皮肤。	后端 RNG、概率、保底和历史。
客户端配置	使用旧配置绕过成本。	后端按 config_version 校验；新 trip 使用 live config。

24.1 风险规则

规则	建议阈值	处理方式
tick 最大时长	max_online_tick_seconds = 15	超过部分截断，不补偿。
tick 最小间隔	同一 trip 每秒最多 2 次	Redis 限流，返回 429。
每分钟有效驾驶时间	不超过 70 秒有效 tick	超过写 suspicious_events。
速度理论上限	vehicle.base_speed × 1.35 × route max multiplier	超过记录并 clamp。
离线最低阈值	小于 60 秒不生成报告	更新 last_seen，避免碎片刷报告。
离线上限	8 小时	超过按 8 小时计算。
重复领取	同 report_id claimed=true	幂等返回或 409，不发奖。

24.2 suspicious_events 记录规范

risk_type	触发场景	severity	action_taken
TICK_RATE_LIMITED	drive tick 过频	1	LOG_ONLY / RATE_LIMIT
INVALID_TICK_DURATION	tick 时间为负或过大	2	CLAMP_AND_LOG
SPEED_LIMIT_EXCEEDED	理论速度超过上限	2	CLAMP_AND_LOG
INVALID_MODE	未解锁 mode	2	REJECT
REWARD_DUPLICATE_ATTEMPT	重复领取已 claim 奖励	3	REJECT
GACHA_IDEMPOTENCY_CONFLICT	同幂等键请求体不同	3	REJECT
WALLET_NEGATIVE_ATTEMPT	扣费后余额将为负	4	REJECT_AND_LIMIT

24.3 V1 处理策略

V1 使用 LOG_ONLY、RATE_LIMIT / REJECT、LIMITED 三层策略。高风险玩家先标记为 WATCH 或 LIMITED，限制离线收益、Gacha 频率或后台提示，等待人工复核。

25. Admin Config System

V1 绝不能把路线距离、车辆属性、维护价格、任务奖励、天气权重、Gacha 概率写死在代码中。上线后的主要工作是持续调参，因此后台配置系统需要支持 Draft、Validate、Publish、Rollback、Config checksum、Config audit 和经济模拟器。

配置域	具体配置	校验重点
路线	route_id、起点、终点、总距离、route_type、difficulty、reward_multiplier	距离符合类型范围；route_id 唯一。
路段	segment 顺序、起止距离、terrain、biome、倍率	连续、不重叠、不留空、末段到总距离。
景点	landmark 距离、rarity、required_stop、photo reward	不能超过总距离；强制景点间距合理。
车辆	速度、油箱、油耗、耐久损耗、离线效率、稀有度	速度和油耗在合理范围。
维护	加油、洗车、修理价格与地区倍率	成本不超过玩家日收入合理比例。
路线解锁	unlock_cost_stamps、Trip Prep Fee	符合 clamp；Tutorial=0。
天气	天气权重、倍率、seed_salt、refresh_distance	权重有效；倍率不极端。
每日登录	7 日奖励	Day 7 完整 Stamp 每周最多 1 个。
任务	目标事件、目标值、奖励	每日任务 10-20 分钟可完成。
Gacha	概率、保底、奖池、重复转化	概率和=100%；保底可模拟。
离线	max_offline_hours、offline_coin_per_km、base_speed	离线收益低于在线。

25.1 配置发布流程

Draft Config -> Validate Schema -> Validate Route Continuity -> Validate Economy Formulas
-> Simulate 7-day / 30-day Player Cohorts -> Review Diff -> Publish Live Version
-> Monitor Metrics -> Rollback if needed

25.2 配置校验规则

- 路线 segment 连续覆盖 total_distance_km。
- Tutorial 80-120 km, Short 120-300 km, Medium 300-800 km, Long 800-1500 km, Epic 1500-3000 km。
- 解锁成本按公式后落在 route_type clamp 区间；Tutorial=0。
- Trip Prep Fee 使用 clamp(round(distance × 0.35), 20, 300) 或后台明确 override 且有理由。
- Gacha 概率权重可归一化后等于 100%；保底阈值大于 0；池中至少有 Rare 以上物品。
- 离线 coin/km < 在线 coin/km，离线 token 门槛高于在线。

25.3 经济模拟器

```
function simulate_economy(days, player_archetype, config_version):
  state = init_new_player_state()
  for day in 1..days:
    simulate_online_distance(state, sampled_online_minutes)
    simulate_offline_distance(state, sampled_offline_hours)
```

```
    apply_maintenance_if_needed(state)
    complete_daily_quests_by_probability(state)
    claim_daily_login(state)
    unlock_route_if_possible(state)
    optionally_spend_tokens_on_gacha(state)
    record_daily_snapshot(state)
    return average_coin_balance, token_balance, routes_unlocked, maintenance_cost_ratio, stamps_earned, gacha_pulls
```

25.4 发布前 checklist

- ☐ 所有路线 segment 连续且覆盖全程
- ☐ Tutorial 路线 80-120 km 且免费
- ☐ 所有路线 unlock cost 符合 clamp
- ☐ 所有 Trip Prep Fee 不超过 300 Road Coins
- ☐ 天气权重有效，极端天气不会连续过多
- ☐ 车辆油耗不会让第一趟教程中途断油
- ☐ 维护价格不会超过每日收入合理比例
- ☐ Gacha 概率总和正确，保底逻辑通过模拟
- ☐ Daily Login 不破坏 Stamps 稀缺性
- ☐ 7 日/30 日经济模拟无明显通胀或供断
- ☐ 关键埋点事件已同步更新
- ☐ checksum 已生成，发布人和审批记录完整

26. Config Version 数据版本控制

配置版本控制用于保证线上稳定性和可回滚性。V1 的核心约定是：配置可以变化，但进行中的旅程不能因为配置变化而变得不可解释。

规则	说明
每次改配置都生成版本	不要直接改 live 行；从 live 复制为 DRAFT。
新玩家使用 live version	创建玩家或初始化默认路线时读取最新 LIVE。
新 trip 使用 live version	POST /routes/start 将 live config_version 写入 player_trips。
进行中的 trip 使用开始时版本	模拟引擎根据 trip.route_config_version 加载路线、天气、奖励。
旧配置不删除	旧版本标记 DEPRECATED，供历史 trip 和审计查询。
回滚生成新版本	复制旧版本为新的 LIVE，而不是直接复活旧行。

26.1 迁移策略

无需迁移的改动包括任务奖励、Gacha banner、未来路线；新旅程生效的改动包括路线距离、路段、天气、景点；需要玩家状态迁移的改动包括货币类型、删除车辆、道具结构调整。第三类必须提供 migration preview 和 migration_log。

26.2 migration preview

迁移预览必须回答：多少玩家拥有受影响车辆/道具；多少玩家正在进行受影响路线；需要补偿多少资源；是否会导致余额为负；是否影响 pending offline_reports；是否可分批执行和回滚。

26.3 rollback

回滚流程：选择目标旧版本 -> 完整配置校验 -> 生成新 config_version -> 当前 live 标记 DEPRECATED -> 新版本标记 LIVE -> 新旅程使用回滚版本，旧旅程仍按开始版本继续 -> 发送 config_rollback analytics 事件。

27. Analytics 数据埋点

Analytics 的目标是让团队判断 V1 核心循环是否成立：玩家是否完成第一趟、是否领取离线报告、是否继续解锁路线、是否拍照收集、经济是否健康。V1 必须以后端埋点为主。

事件名	触发时机	核心字段	分析用途
tutorial_start	开始教程	player_id, session_id, tutorial_state, config_version	判断教程入口是否正常
tutorial_complete	教程完成	duration_seconds, first_route_id	教程完成率和耗时
auto_driving_unlocked	解锁 Auto Driving	trip_id, distance_km, elapsed_seconds	验证解锁时机
first_trip_complete	第一趟完成	route_id, duration_seconds, photos_taken	新手体验核心漏斗
route_unlocked	路线解锁	route_id, cost_stamps, balance	Stamps 成本是否合理
route_started	路线开始	route_id, route_type, prep_fee, vehicle_def_id	路线选择和出发成本
route_completed	路线完成	route_id, duration, online/offline distance	路线完成时长和掉点
offline_report_generated	离线报告生成	offline_seconds, distance, pending_rewards, forced_stop_reason	离线系统触发量
offline_report_claimed	离线报告领取	report_id, rewards, claim_delay_seconds	回归领取率
vehicle_refueled	加油	vehicle_id, missing_fuel, cost	油耗/加油负担
vehicle_cleaned	洗车	clean_points, cost, cleanliness_after	清洁系统是否过贵
vehicle_repaired	修理	repair_points, cost, durability_after	耐久系统负担
gacha_pull	抽卡	banner_id, pull_count, cost_tokens, rarity_results	Gacha 参与和概率校验
photo_taken	拍照	landmark_id, quality_score, first_photo, rewards	照片系统有效性
daily_login_claimed	每日登录领取	cycle_day, reward	登录奖励参与度
quest_completed	任务完成	quest_id, period_key, duration	任务难度
wallet_currency_changed	钱包变化	currency, amount, reason,	经济通胀和异常

事件名	触发时机	核心字段	分析用途
		balance_after	
trip_abandoned	放弃旅程	route_id, current_distance, reason	路线流失点
stopped_at_landmark	景点停靠	landmark_id, distance_km	强制停靠体验
stopped_low_fuel	低油停靠	trip_id, route_id, distance_km	油量是否像体力

27.1 统一事件字段

字段	类型	说明
event_id	uuid	事件唯一 ID
event_name	text	事件名
event_time	timestamp	服务器时间
player_id	uuid	可为空
session_id	text	客户端会话 ID
trip_id	uuid	如果与旅程相关
route_id	uuid	如果与路线相关
config_version_id	uuid	当前事件使用的配置版本
server_version	text	后端版本
payload	json	事件特有字段

27.2 核心指标与判断方法

问题	观察指标	判断方式	调整方向
教程太慢	tutorial_complete duration, completion rate	中位耗时过长或停在 HOLD_TO_DRIVE_REQUIRED。	缩短第一段距离，提前解锁 Auto。
路线太长	route_completed rate, route_drop_off_point	某类路线完成率低、耗时远高预期。	降低距离、提高离线效率、增加中途奖励。
维护太贵	average coins balance, refuel/repair cost ratio	玩家低油/低耐久且余额不足。	降低维护成本或提高路线完成奖励。
离线系统无效	offline_report_generated/claimed	报告生成少或领取率低。	提高报告 UI 反馈，保证离线收益可感知。
经济通胀	wallet_currency_changed, balance trend	Road Coins 或 Tokens 余额持续累积。	增加非强制 sinks 或下调 sources。
Stamps 太难	first_route_unlock_rate, stamp_fragment balance	碎片长期不足且路线未解锁。	提高周任务/相册奖励，降低 Short 成本。
Gacha 误解为进度墙	gacha_pull 与 route_unlock 相关性	未抽卡玩家路线进度明显受限。	确保路线解锁只依赖 Stamps/玩法奖励。

28. 技术选型建议

V1 推荐技术栈应服务于快速、稳定、可调参、可审计。不要为了架构好看而提前拆复杂微服务，也不要一开始使用 Kubernetes。

层级	推荐方案	原因
Backend	Node.js + NestJS + TypeScript	模块化清晰，适合 REST API、后台、任务事件和类型约束。
Database	PostgreSQL	强事务、外键、JSONB、索引、审计能力适合钱包和配置版本。
Cache	Redis	限流、幂等短缓存、分布式锁、热点配置缓存。
Analytics	PostHog / Firebase	快速接入漏斗、留存、事件分析；后端同步落表。
Hosting	Render / Railway / Supabase / AWS / GCP	早期托管服务降低运维复杂度。
Admin Panel	Retool 或自研后台	Retool 快速搭建配置/查询；核心发布流程逐步自研。
Client	Unity 或 Godot	Unity 生态成熟；Godot 对像素风轻量项目友好。

28.1 为什么不一开始微服务

V1 玩家规模、团队规模和系统复杂度尚未证明需要拆分；钱包、旅程、车辆、任务之间事务耦合高，过早拆服务会增加分布式事务复杂度。

28.2 为什么不一开始 Kubernetes

Kubernetes 适合规模化编排，但早期会增加部署、监控、安全、成本和运维难度。V1 应先用托管方案快速验证核心循环。

29. V1 开发阶段路线图

开发顺序应按可运行核心循环逐步扩展，而不是先做所有内容。

Phase	目标	后端功能	数据表	测试重点	交付物
Phase 1	基础账号、玩家存档、车辆和路线	Auth、Player、Route、Vehicle 基础；默认车辆；教程路线可见	players, player_vehicles, route tables	鉴权、默认存档、路线连续性	可登录、可查看路线、可选择车辆。
Phase 2	在线行驶和 Trip Simulation Engine	Trip 创建、drive tick、距离/油耗/forced stop	player_trips, wallet tables	tick 公式、油量限制、并发锁	可在线驾驶并看到距离推进。

Phase	目标	后端功能	数据表	测试重点	交付物
Phase 3	Auto Driving 和 Hold to Boost	三种驾驶模式、mode 校验、解锁状态	players, player_trips	mode 解锁、倍率、非法 mode	Hold/Auto/Boost 三模式可用。
Phase 4	离线推进和 Travel Report	离线模拟、report 生成、pending 奖励、领取	offline_reports	离线上限、油量、景点、重复领取	回归生成 Travel Report 并可领取。
Phase 5	经济系统和钱包交易	全货币余额、流水、sources/sinks	wallet_balances, wallet_transactions	并发扣费、幂等、负余额防护	所有奖励/扣费可审计。
Phase 6	车辆维护	加油、洗车、修理公式与状态影响	player_vehicles, wallet_transactions	价格公式、余额不足、状态边界	维护循环可玩且不阻断首日。
Phase 7	景点和照片系统	景点停靠、照片质量、首次/重复奖励	landmarks, player_photos	提前拍照、重复奖励限制	可收集照片卡并触发奖励。
Phase 8	路线解锁系统	Stamps 永久解锁, Trip Prep Fee	player_unlocked_routes	成本 clamp、不重复收费	完成路线后能解锁下一批路线。
Phase 9	Gacha 和商店	Banner、奖池、概率、保底、Blueprints	gacha tables	概率统计、保底、幂等	抽车/皮肤收藏可用但不影响进度。
Phase 10	每日登录和任务	7 日登录、每日/每周任务、事件驱动	daily_login, quest tables	period key、防重复领取	日常循环可用。
Phase 11	后台配置和 Analytics	Draft/Validate/Publish/Rollback、事件落表	config_versions, analytics_events	配置校验、发布锁、事件完整性	策划可安全调参，团队可看指标。
Phase 12	测试、调参和软上线	全链路测试、经济模拟、压力测试、软上线 cohort	所有核心表	回归、并发、概率、离线、配置、埋点	可控范围软上线并进入调参周期。

30. 测试方案

V1 测试目标是保证模拟正确、钱包正确、配置安全、经济可控、关键埋点完整。

测试类型	测试重点	例子	通过标准
单元测试	纯函数公式和边界	distance_gain、fuel_used、photo_quality、unlock_cost、trip_prep_fee	给定输入输出精确匹配。
集成测试	API + DB 事务	start -> drive tick -> forced stop -> refuel -> continue	状态和钱包流水一致。
经济模拟测试	7 日/30 日 sources/sinks	不同玩家模型下余额走势	无明显通胀，无首日供断。
离线模拟测试	server time、上限、油量、景点、终点	离线 12 小时仅按 8 小时	报告正确且不重复。
钱包并发测试	同时扣费/发奖	并发 20 个 refuel 或 claim report	无负余额、无重复流水。
Gacha 概率测试	概率分布和保底	模拟大量抽卡，检查 10/30/80 保	误差在阈值内，保底必触发。

测试类型	测试重点	例子	通过标准
		底	
配置校验测试	路线和经济配置	segment 不连续、概率不为 100%、Tutorial 收费	校验失败并写 validation_report。
反作弊测试	非法客户端请求	超频 tick、非法 mode、重复领取、篡改 route_id	拒绝或 clamp 并写 suspicious_events。
数据埋点测试	事件完整性	完成第一趟路线应发完整事件	字段完整，player/trip/config 对齐。
软上线测试	真实 cohort 指标	小流量新玩家 3-7 天观察	关键指标达到门槛后扩大测试。

30.1 关键测试用例示例

Test: drive tick should stop at required landmark
Given: trip.current_distance_km = 39.9, next_required_landmark.distance_km = 40.0, distance_raw = 0.5
When: simulate_online_tick()
Then: distance_gain_km = 0.1, trip.status = FORCED_STOP, reason = LANDMARK_REQUIRED, no distance beyond landmark is rewarded

Test: offline report claim is idempotent
Given: offline_report.claimed = false, road_coins_pending = 100
When: POST /trip/claim-offline-report with same idempotency_key twice
Then: first request creates one wallet transaction +100; second returns same result; balance increases once

Test: pending offline report prevents duplicate generation
Given: trip has unclaimed offline_report
When: GET /player/state called three times
Then: no new report inserted; same pending report returned

31. 风险清单与解决方案

风险	风险描述	影响	解决方案	监控指标
路线太长	Short/Medium 完成耗时超过玩家预期。	留存下降，route_completed 低。	缩短前期路线，提高中途奖励，增加离线反馈。	route_completion_time, route_drop_off_point
教程过慢	第一趟教程步骤太多或距离过长。	Tutorial Completion Rate 下降。	教程 80-120 km，首个景点提前，Auto 早解锁。	tutorial_complete_rate, tutorial_duration
Auto Driving 解锁太晚	玩家长时间 Hold to Drive。	操作疲劳，早期流失。	教程前 1/3 解锁 Auto。	auto_driving_unlocked_distance
油量像体力系统	没油后只能等待或频繁付费。	玩家认为被卡进度。	第一趟低油耗，加油只消耗可获得 Road Coins，不做等待恢复。	stopped_low_fuel, refuel_cost_ratio
离线收益太高	玩家只离线即可获得大量资源。	在线驾驶失去意义，经济通胀。	离线 4 Coins/km，低于在线 10 Coins/km；8 小时上限。	offline/online reward ratio
维护成本太高	修理/加油/洗车吃掉大部分收入。	路线循环断裂。	维护成本占每日收入设上限，初期损耗低。	average coins balance, maintenance cost ratio
Gacha 被认为 pay-to-progress	抽到高稀有车显著加速核心进度。	玩家认为不抽无法玩。	Gacha 只影响收藏/外观/轻微效率，不用于路线解锁。	route progress by gacha/non-gacha
Stamps 太难获得	路线解锁慢，玩家无下一目标。	First Route Unlock Rate 低。	Short 成本 1-2，周任务/相册给 Stamps，日登录给碎片。	stamp balance, route_unlock_rate

风险	风险描述	影响	解决方案	监控指标
钱包重复发奖	重试或并发导致奖励多发。	经济被破坏，客服难处理。	幂等键、唯一索引、同事务流水。	duplicate tx count
离线重复计算	多次打开 state 生成多份报告。	离线收益翻倍。	已有 pending report 时不再生成；锁 trip；更新 last_simulated_at。	offline_reports per trip per hour
玩家跳过景点	离线或在线越过 required landmark。	照片系统失效。	forced stop clamp 到 landmark，complete-landmark 校验。	stopped_at_landmark rate
配置误发布	错误倍率/成本上线。	大面积经济异常。	Draft/Validate/Simulate/Publish/Rollback。	config publish errors, economy spike
Gacha 概率错误	概率和不对或保底失效。	玩家信任风险。	发布前概率模拟，抽卡历史审计。	rarity distribution
数据库并发问题	两个请求同时推进/扣费。	状态错乱、重复奖励。	SELECT FOR UPDATE、唯一幂等、事务边界。	deadlock/retry count
客户端作弊	篡改时间、距离、奖励。	经济和进度异常。	后端权威计算、风险日志、限流。	suspicious_events count
后台权限问题	非授权人员发布配置或看玩家数据。	安全和运营事故。	RBAC、审计日志、双人审批。	admin audit anomalies
Analytics 缺事件	软上线无法判断问题。	调参靠猜。	事件 checklist、服务端事件落表、测试验证。	event coverage rate
旧配置被删除	历史 trip 无法复现。	模拟/客服/审计失败。	旧版本 deprecated 不删除。	missing config reference
route segment 不连续	模拟引擎找不到路段或倍率错误。	行驶卡住、异常奖励。	配置校验覆盖连续性和总长。	route validation failures
经济通胀	sources 远大于 sinks。	货币失去意义。	7/30 日模拟，调低收益或增加非强制 sinks。	avg balance trend
离线报告反馈弱	玩家回归不知道发生了什么。	离线留存价值下降。	Travel Report 展示距离、天气、景点、奖励、停靠原因。	offline_claim_rate
首日 Gacha 过强	玩家把 Token 全抽掉后分散核心循环。	路线/维护体验被抽卡抢走。	首日 Gacha 引导延后，抽卡只用 Tokens 不用 Stamps。	first_day_gacha_spend

32. 最终 Backend Checklist

以下 checklist 用于开发、测试、软上线前逐项确认。每项建议绑定负责人和验收用例。

32.x 玩家系统

☐ Auth token 解析和 player_id 绑定完成 ☐ 新玩家初始化默认车辆、钱包和教程状态 ☐ tutorial_state 状态机只允许合法转移 ☐ risk_status 可读可写，并在后台可查询 ☐ player.last_seen_at 使用服务器时间更新

32.x 钱包系统

☐ wallet_balances 支持所有 V1 货币 ☐ wallet_transactions 对所有变化留痕 ☐ 扣费使用 SELECT FOR UPDATE 锁余额 ☐ 余额不足回滚并返回明确错误 ☐ 所有发奖/扣费接口具备幂等键 ☐ 流水支持 source_type/source_id 查询

32.x 路线系统

☐ route_definitions 支持 route_type、difficulty、config_version ☐ route_segments 连续性校验完成 ☐ landmarks 距离合法且属于路线 ☐ Tutorial 路线 80-120 km 且免费 ☐ Short/Medium/Long/Epic 距离范围符合要求 ☐ route_config_version 在 trip 开始时写入

32.x 教程系统

☐ 3 个起点 × 3 个目的地配置完成 ☐ 第一趟路线免费且不要求 Gacha ☐ Hold to Drive 引导完成 ☐ Auto Driving 解锁时机明确 ☐ Hold to Boost 转化完成 ☐ 第一个景点和第一次拍照完成 ☐ FULL_SYSTEM_UNLOCKED 后开放完整系统

32.x 客户端体验层

☐ 主屏幕车辆持续前进表现完成 ☐ route segment 背景切换完成 ☐ 天气和昼夜色调完成 ☐ Route Board 展示剩余距离/天气/景点/进度 ☐ Travel Report 视觉化展示距离/奖励/损耗/停靠原因 ☐ 客户端预测动画会被后端状态纠正

32.x 车辆系统

☐ vehicle_definitions 属性完整 ☐ player_vehicles 保存油量、耐久、清洁度、皮肤、等级 ☐ 当前选中车辆唯一约束完成 ☐ 旅程中换车规则明确并校验

32.x 维护系统

☐ fuel_used 公式实现 ☐ cleanliness_loss 公式实现 ☐ durability_loss 公式实现 ☐ refuel/clean/repair cost 公式实现 ☐ 油量 0 阻止继续推进 ☐ 清洁度低不阻止行驶 ☐ 耐久低温和影响速度/油耗/离线效率

32.x 在线驾驶

☐ drive-tick 后端权威计算 ☐ tick duration 使用服务器时间 ☐ max_online_tick_seconds clamp ☐ mode_multiplier 配置化 ☐ segment/weather multiplier 实现 ☐ forced stop 检查完成 ☐ 在线 Road Coins 和 Token meter 正确 ☐ tick 幂等和限流完成

32.x 离线驾驶

☐ 不信任客户端时间 ☐ offline_seconds 使用 server_now 计算 ☐ max_offline_hours = 8 ☐ base_offline_speed_kmph = 30 配置化 ☐ 油量限制距离实现 ☐ 强制景点停靠实现 ☐ 终点停靠实现 ☐ 离线收益低于在线收益 ☐ 离线模拟不会重复计算

32.x Travel Report

☐ offline_reports 独立表实现 ☐ report pending 奖励不直接入账 ☐ 已有 pending report 时不生成新 report ☐ claim-offline-report 幂等 ☐ claimed/claimed_at 正确更新 ☐ report 展示距离、奖励、天气、损耗、停靠原因

32.x 景点与照片

□ landmark 数据结构完整 □ photo_quality_score 实现 □ 清洁度/天气/昼夜影响照片奖励 □ 首次拍照奖励和重复拍照奖励区分 □ 每个 landmark 首次照片唯一约束 □ 相册完成奖励可发放 □ 防重复刷资源逻辑完成

32.x 天气与昼夜

□ 6 种天气配置完成 □ weather_seed 和 weather_bucket 可复现 □ 离线模拟与在线天气一致 □ 天气倍率影响速度、油耗、清洁、耐久、照片 □ game_time_speed_multiplier = 10 配置化 □ Dawn/Day/Sunset/Night 计算正确

32.x 路线解锁经济

□ Tutorial 免费 □ Souvenir Stamps 永久解锁路线 □ 解锁后不重复收 Stamps □ Trip Prep Fee 只收 Road Coins 且封顶 □ Stamp Fragments 10:1 合成 Stamps □ 完成路线、相册、周任务可获得 Stamps □ Daily Login Day 7 完整 Stamp 每周最多 1 个

32.x Gacha

□ Gacha 不影响路线核心进度 □ 概率 Common 70 / Rare 22 / Epic 7 / Legendary 1 □ 10/30/80 抽保底实现 □ 单抽 20 Tokens，十连 180 Tokens □ 重复物品转 Blueprints □ gacha_history 完整记录 □ 抽卡后端 RNG，客户端不能传结果

32.x 每日登录与任务

□ 7 日循环奖励配置完成 □ period_key 时区策略明确 □ daily_login_state 防重复领取 □ quest_definitions 支持每日/每周 □ player_quest_progress 事件驱动更新 □ quest claim 幂等 □ 任务奖励不破坏经济

32.x Admin Config

□ Draft/Validate/Publish/Rollback API 完成 □ 配置 checksum 生成 □ 发布审批和审计记录 □ 路线连续性校验 □ Gacha 概率校验 □ 经济 7/30 日模拟 □ 发布前 checklist 自动化

32.x Config Version

□ 每次配置修改生成新版本 □ 新玩家使用 live version □ 新 trip 写入 live route_config_version □ 进行中 trip 使用开始版本 □ 旧版本 deprecated 不删除 □ migration preview 和 migration_log 完成 □ 回滚生成新 live version

32.x Analytics

□ 必要事件全部由后端发出 □ wallet_currency_changed 覆盖所有钱包变化 □ route_drop_off_point 可分析 □ offline report 漏斗可分析 □ tutorial 漏斗可分析 □ Gacha 结果可审计 □ PostHog/Firebase 或本地事件表接入完成

32.x 反作弊

□ 客户端时间不被信任 □ 客户端距离不被信任 □ 客户端奖励不被信任 □ 客户端抽卡结果不被信任 □ tick 限流实现 □ 理论速度上限检查 □ suspicious_events 记录完成 □ risk_status 后台可查询 □ V1 不自动封号，先标记和限制

32.x 测试

□ 单元测试覆盖核心公式 □ 集成测试覆盖核心循环 □ 钱包并发测试通过 □ 离线模拟测试通过 □ Gacha 概率测试通过 □ 配置校验测试通过 □ 反作弊测试通过 □ 埋点测试通过

32.x 软上线

□ 小流量 cohort 创建 □ D1/D7 指标看板完成 □ 教程漏斗看板完成 □ 经济余额看板完成 □ 风险事件看板完成 □ 回滚流程演练完成 □ 客服查询玩家 trip/wallet/report 工具完成

33. Change Log / 本次优化说明

优化项	说明
修复中文排版和断字	重写并整理原报告中的断字、缺字、乱码、奇怪空格、表格截断和句子不完整问题。
统一术语	统一旅游模拟器 / Travel Simulator、Trip Simulation Engine、Economy Ledger、Route Config System、Road Coins、Travel Tokens、Souvenir Stamps、Stamp Fragments、Blueprints、Travel Report、Curated Route Packs、Hold to Drive、Auto Driving、Hold to Boost、Trip Prep Fee、Forced Stop。
统一经济数值	在线 1 km = 10 Road Coins；离线 1 km = 4 Road Coins；在线 10 km = 1 Token；离线 20 km = 1 Token；单抽 20 Tokens；十连 180 Tokens；Trip Prep Fee 公式全篇一致。
补充客户端主界面体验层	新增主屏幕驾驶表现、像素背景切换、天气/昼夜表现、Route Board、Travel Report UI、景点停靠与照片卡、客户端预测动画与后端权威状态关系。
补充 V1 Scope Control	新增 P0/P1/P2 范围控制和不进入 V1 内容。
补充 offline report 防重复生成规则	明确已有未领取 report 时 GET /player/state 不生成新 report，而是返回现有 pending report。
调整 Daily Login 的 Stamp 风险	Day 7 可给 1 Souvenir Stamp，但每周最多 1 个；核心 Stamps 仍来自路线完成、相册完成和周任务。
强化 P0/P1/P2 优先级	在模块、scope、路线图和 checklist 中明确优先级。
强化测试和风险说明	补充测试类型、关键用例、20+ 风险和监控指标。

34. 最终架构师建议

V1 最重要不是内容多，而是核心循环稳定。对于《旅游模拟器 / Travel Simulator》，第一版的胜负不在于是否有真实地图、真实导航或复杂多人，而在于玩家是否愿意完成第一趟、回来领取 Travel Report、维护车辆、继续下一段旅程、收集下一张照片卡。

1. **先做 Trip Simulation Engine。** 在线行驶、离线推进、油耗、景点停靠、奖励和路线完成都依赖它。引擎不稳，增加车辆、路线、Gacha 都没有意义。
2. **钱包系统必须严谨。** Road Coins、Travel Tokens、Souvenir Stamps、Stamp Fragments、Blueprints 都必须有余额、有流水、有 reason、有 source、有幂等。
3. **路线系统必须配置化。** Curated Route Packs 才能控制路线长度、景点节奏、天气、背景、奖励和难度。
4. **离线系统要克制。** 离线用于留存和旅行感，不是替代在线；离线收益必须低于在线，离线报告必须 pending 领取。
5. **Gacha 只做收藏和外观。** Gacha 不决定路线能否继续，不能成为路线解锁 paywall。

- 6. **第一趟教程必须短、顺、有正反馈。** 80-120 km、免费、快速引导 Hold to Drive、Auto Driving、第一个景点和第一次拍照。
- 7. **不要一开始做真实地图。** 真实地图和真实导航会放大复杂度，却不能更快验证核心循环。
- 8. **所有数值必须后台配置。** 路线距离、油耗、维护价格、离线时长、任务奖励、Gacha 概率、天气权重和解锁成本都必须可调。
- 9. **每个关键行为必须埋点。** 没有 Analytics，就无法判断教程、路线、维护、离线和经济是否健康。
- 10. **V1 成功标准很明确。** 玩家愿意回来领取离线报告，愿意继续下一段旅程，愿意维护车辆，愿意解锁下一条路线，愿意收集下一张照片卡。

附录 A：默认参数表

参数	推荐值	说明
max_offline_hours	8	离线推进上限。
base_offline_speed_kmph	30	基础离线速度。
online_coin_per_km	10	在线每公里 Road Coins。
offline_coin_per_km	4	离线每公里 Road Coins，低于在线。
online_token_km	10	在线每 10 km 产 1 Travel Token。
offline_token_km	20	离线每 20 km 产 1 Travel Token。
game_time_speed_multiplier	10	昼夜速度倍率。
max_online_tick_seconds	15	在线 tick 最大有效秒数。
min_offline_report_seconds	60	生成离线报告最低离线时间。
trip_prep_fee_min	20	Trip Prep Fee 下限。
trip_prep_fee_max	300	Trip Prep Fee 上限。
Hold to Drive multiplier	1.00	标准在线驾驶。
Auto Driving multiplier	0.85	自动驾驶低于手动但不惩罚。
Hold to Boost multiplier	1.10	短时加速，避免 1.25 带来膨胀。
Gacha rates	Common 70%, Rare 22%, Epic 7%, Legendary 1%	收藏概率。
Pity rules	10 Rare, 30 Epic, 80 Legendary	保底规则。
Gacha single / ten	20 / 180 Travel Tokens	十连有 10% 折扣。
Stamp Fragment conversion	10 = 1 Souvenir Stamp	碎片转换。
Tutorial route	80-120 km	第一趟路线。
Short route	120-300 km	早期日常路线。
Medium route	300-800 km	核心中期路线。

参数	推荐值	说明
Long route	800-1500 km	长期目标。
Epic route	1500-3000 km	大型路线包。

附录 B：术语表

中文术语	英文术语	定义
旅游模拟器	Travel Simulator	本项目名称。
V1	V1	第一版可落地版本。
旅程模拟引擎	Trip Simulation Engine	后端权威模拟距离、消耗、停靠和奖励的核心模块。
经济流水系统	Economy Ledger	记录所有货币变化的余额与流水系统。
路线配置系统	Route Config System	管理路线、路段、景点、天气、奖励和版本。
公路金币	Road Coins	基础软货币，用于维护、出发费和日常消耗。
旅行代币	Travel Tokens	收藏/Gacha 货币。
纪念印章	Souvenir Stamps	路线永久解锁货币。
印章碎片	Stamp Fragments	10 个合成 1 个 Souvenir Stamp。
蓝图碎片	Blueprints	重复物品补偿和兑换材料。
离线旅行报告	Travel Report	离线推进后生成的 pending 奖励报告。
手工配置路线包	Curated Route Packs	策划配置的路线、路段、景点、天气和背景。
长按驾驶	Hold to Drive	玩家主动按住推进的驾驶模式。
自动驾驶	Auto Driving	无需持续按住的驾驶模式，倍率 0.85。
长按加速	Hold to Boost	短时加速模式，倍率 1.10。
旅程准备费	Trip Prep Fee	每次出发的 Road Coins 成本。
强制停靠	Forced Stop	因低油、景点、终点或异常而停止推进。

附录 C：核心公式索引

公式	章节	用途
unlock_cost	9	路线解锁成本。

公式	章节	用途
trip_prep_fee	9 / 10	出发准备费。
upgrade_cost	10 / 11	车辆升级成本。
distance_gain	13 / 23	在线距离推进。
offline_seconds	14	离线时间计算。
offline_speed_kmph	14	离线速度。
fuel_used	12 / 23	燃油消耗。
cleanliness_loss	12 / 23	清洁度损耗。
durability_loss	12 / 23	耐久损耗。
photo_quality_score	16	照片质量。
weather_seed / weather_bucket	17	天气复现。
current_game_minutes	18	昼夜计算。
online_road_coins	10 / 23	在线 Road Coins 奖励。
offline_road_coins	10 / 23	离线 Road Coins 奖励。